SISTEMAS MULTIAGENTE E APRENDIZADO POR REFORÇO APLICADOS A META-HEURÍSTICAS PARA OTIMIZAÇÃO DO PROBLEMA DE ROTEAMENTO DE VEÍCULOS COM JANELAS DE TEMPO

Hugo de Barros Araujo

Projeto de Graduação apresentado ao Curso de Engenharia Eletrônica e de Computação da Escola Politécnica, Universidade Federal do Rio de Janeiro, como parte dos requisitos necessários à obtenção do título de Engenheiro.

Orientador: Pedro Henrique Gonzalez Silva

Rio de Janeiro

Fevereiro de 2026

SISTEMAS MULTIAGENTE E APRENDIZADO POR REFORÇO APLICADOS A META-HEURÍSTICAS PARA OTIMIZAÇÃO DO PROBLEMA DE ROTEAMENTO DE VEÍCULOS COM JANELAS DE TEMPO

Hugo de Barros Araujo

PROJETO DE GRADUAÇÃO SUBMETIDO AO CORPO DOCENTE DO CURSO DE ENGENHARIA ELETRÔNICA E DE COMPUTAÇÃO DA ESCOLA POLITÉCNICA DA UNIVERSIDADE FEDERAL DO RIO DE JANEIRO COMO PARTE DOS REQUISITOS NECESSÁRIOS PARA A OBTENÇÃO DO GRAU DE ENGENHEIRO ELETRÔNICO E DE COMPUTAÇÃO

Autor:

_____
Hugo de Barros Araujo

Orientador:

_____
Pedro Henrique Gonzalez Silva, Ph. D.

Examinador:

_____
Prof Frances Elizabeth Allen, D. Sc.

Examinador:

_____
Prof. Alan Jay Perlis, D. E.

Rio de Janeiro

Fevereiro de 2026

Declaração de Autoria e de Direitos

Eu, *Hugo de Barros Araujo*, autor da monografia *SISTEMAS MULTIAGENTE E APRENDIZADO POR REFORÇO APLICADOS A META-HEURÍSTICAS PARA OTIMIZAÇÃO DO PROBLEMA DE ROTEAMENTO DE VEÍCULOS COM JANELAS DE TEMPO*, subscrevo para os devidos fins, as seguintes informações:

1. O autor declara que o trabalho apresentado na disciplina de Projeto de Graduação da Escola Politécnica da UFRJ é de sua autoria, sendo original em forma e conteúdo.

2. Excetuam-se do item 1. eventuais transcrições de texto, figuras, tabelas, conceitos e idéias, que identifiquem claramente a fonte original, explicitando as autorizações obtidas dos respectivos proprietários, quando necessárias.

3. O autor permite que a UFRJ, por um prazo indeterminado, efetue em qualquer mídia de divulgação, a publicação do trabalho acadêmico em sua totalidade, ou em parte. Essa autorização não envolve ônus de qualquer natureza à UFRJ, ou aos seus representantes.

4. O autor pode, excepcionalmente, encaminhar à Comissão de Projeto de Graduação, a não divulgação do material, por um prazo máximo de 01 (um) ano, improrrogável, a contar da data de defesa, desde que o pedido seja justificado, e solicitado antecipadamente, por escrito, à Congregação da Escola Politécnica.

5. O autor declara, ainda, ter a capacidade jurídica para a prática do presente ato, assim como ter conhecimento do teor da presente Declaração, estando ciente das sanções e punições legais, no que tange a cópia parcial, ou total, de obra intelectual, o que se configura como violação do direito autoral previsto no Código Penal Brasileiro no art.184 e art.299, bem como na Lei 9.610.

6. O autor é o único responsável pelo conteúdo apresentado nos trabalhos acadêmicos publicados, não cabendo à UFRJ, aos seus representantes, ou ao(s) orientador(es), qualquer responsabilização/ indenização nesse sentido.

7. Por ser verdade, firmo a presente declaração.

Hugo de Barros Araujo

UNIVERSIDADE FEDERAL DO RIO DE JANEIRO
Escola Politécnica - Departamento de Eletrônica e de Computação
Centro de Tecnologia, bloco H, sala H-217, Cidade Universitária
Rio de Janeiro - RJ CEP 21949-900

Este exemplar é de propriedade da Universidade Federal do Rio de Janeiro, que poderá incluí-lo em base de dados, armazenar em computador, microfilmar ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es).

AGRADECIMENTO

Ao povo brasileiro, que financia esta universidade e tornou possível minha formação em uma instituição de excelência como a UFRJ. Que a educação pública, gratuita e de qualidade continue sendo um pilar de transformação, tornando-se cada vez mais inclusiva e acessível a todos.

À minha família, a base de tudo o que sou. Um agradecimento profundo e especial aos meus pais, José Antonio e Rosália, que não mediram esforços para que eu chegasse até aqui. O apoio incondicional, os sacrifícios silenciosos e o incentivo constante foram o combustível necessário para concluir esta jornada. Vocês são meu maior exemplo de dedicação e amor.

Ao meu tio e padrinho, Lorran, quem mais me ajudou a despertar o gosto pela ciência.

À minha namorada, Eduarda, parceira de todas as horas. Obrigado por compartilhar a vida comigo, por deixar tudo sempre leve e alegre quando estou com você.

RESUMO

Este trabalho aborda a resolução do Problema de Roteamento de Veículos com Janelas de Tempo (*Vehicle Routing Problem with Time Windows* - VRPTW), um desafio de otimização combinatória classificado como *NP-hard*, cujas aplicações práticas são fundamentais para a eficiência logística e redução de custos operacionais. O objetivo central é o desenvolvimento de um sistema híbrido que integra metaheurísticas clássicas a um Sistema Multiagente (*Multi-Agent System* - MAS) orquestrado por Aprendizado por Reforço (*Reinforcement Learning* - RL). Inicialmente, foram implementados de forma independente os algoritmos de *Simulated Annealing*, *Tabu Search*, e *Genetic Algorithm*, utilizando as instâncias de Solomon como *benchmark* para avaliação de desempenho.

A originalidade da proposta reside na encapsulação desses algoritmos como agentes especialistas dentro do *framework* Mesa, onde a cooperação é estabelecida através de uma base de soluções de elite e da estratégia de *Path Relinking*, que explora trajetórias entre soluções de alta qualidade para evitar ótimos locais. Para elevar a eficiência do sistema, aplicou-se o algoritmo *Q-Learning* como uma hiper-heurística capaz de aprender, de forma autônoma, qual especialista deve ser acionado em diferentes estados do espaço de busca. Os resultados sugerem que a inteligência coletiva e a adaptação dinâmica de parâmetros via RL superam o desempenho individual dos algoritmos, proporcionando soluções otimizadas em tempo computacional viável.

Palavras-chave: Otimização Combinatória, VRPTW, Sistemas Multiagentes, Metaheurísticas, Aprendizado por Reforço, *Path Relinking*.

ABSTRACT

This work addresses the resolution of the Vehicle Routing Problem with Time Windows (VRPTW), a NP-hard combinatorial optimization challenge with critical applications in logistics efficiency and operational cost reduction. The primary objective is to develop a hybrid system that integrates classical metaheuristics within a Multi-Agent System (MAS) orchestrated by Reinforcement Learning (RL). Initially, Simulated Annealing, Tabu Search, and Genetic Algorithms were independently implemented and evaluated using Solomon's benchmark instances to establish performance baselines.

The originality of this approach lies in encapsulating these algorithms as specialized agents within the Mesa framework, where cooperation is established through a shared elite solution pool and the Path Relinking strategy, which explores trajectories between high-quality solutions to circumvent local optima. To enhance system efficiency, the Q-Learning algorithm was applied as a hyper-heuristic capable of autonomously learning which specialist to activate across different states of the search space. Preliminary results suggest that collective intelligence and dynamic parameter adaptation via RL outperform individual algorithmic performance, providing optimized solutions within viable computational time.

Key-words: Combinatorial Optimization, VRPTW, Multi-Agent Systems, Metaheuristics, Reinforcement Learning, Path Relinking.

SIGLAS

GA – *Genetic Algorithm* (algoritmo genético)

I/O – *In/Out* (entrada/saída)

MAS – *Multi-Agent System* (sistema multi-agente)

MAS-RL – *Multi-Agent System with Reinforcement Learning* (sistema multi-agente com aprendizado por reforço)

NP-hard – *Non-deterministic Polynomial-time hard* (problemas de tempo polinomial não-determinístico difícil)

OOP – *Object-Oriented Programming* (programação orientada a objetos)

PR – *Path Relinking* (reconexão por caminhos)

RAM – *Random Access Memory* (memória de acesso aleatório)

RL – *Reinforcement Learning* (aprendizado por reforço)

SA – *Simulated Annealing* (recozimento simulado)

TS – *Tabu Search* (busca tabu)

TSP – *Traveling Salesman Problem* (problema do caixeiro viajante)

UFRJ – Universidade Federal do Rio de Janeiro

VRP – *Vehicle Routing Problem* (problema de roteamento de veículos)

VRPTW – *Vehicle Routing Problem with Time Windows* (problema de roteamento de veículos com janelas de tempo)

Sumário

1 Introdução **1**

1.1 Tema 1

1.2 Delimitação 2

1.3 Justificativa 3

1.4 Objetivos 3

1.5 Metodologia 4

1.6 Descrição 5

2 Fundamentação Teórica **6**

2.1 Definição do problema 6

2.1.1 O Problema de Roteamento de Veículos 7

2.1.2 Metodologia de Avaliação e o Benchmark de Solomon 7

2.2 Seleção do Trio de Meta-heurísticas 9

2.2.1 Recozimento Simulado 10

2.2.2 Busca Tabu 10

2.2.3 Algoritmo Genético 11

2.3 Sistema multi-agente 12

2.3.1 Framework Mesa 12

2.3.2 Estratégia de cooperação: Path Relinking 13

2.4 Aprendizado por Reforço 15

2.4.1 Q-Learning 16

3 Método Proposto **19**

3.1 Etapas iniciais 19

3.2 Implementação de sistema Multi-Agente 21

3.2.1 Arquitetura e Componentes do Sistema 22
3.2.2 Cooperação e a Estratégia de Path Relinking 22
3.3 Hiper-heurística baseada em Aprendizado por Reforço 23
3.3.1 Modelagem do Q-Learning 23
3.3.2 Dinâmica de Cooperação e Atualização 24

4 Experimentos Computacionais 25
4.1 Software e Hardware 25
4.2 Testes com metaheurísticos puros 26
4.2.1 Resultados 28
4.2.2 Análise comparativa 30
4.3 Implementação de sistema Multi-Agente 32
4.3.1 Arquitetura e Componentes do Sistema 32
4.3.2 Cooperação e a Estratégia de Path Relinking 33
4.3.3 Resultados 33
4.3.4 Análise dos resultados 35
4.4 Hiper-heurística baseada em Aprendizado por Reforço 36
4.4.1 Modelagem do Q-Learning 36
4.4.2 Dinâmica de Cooperação e Atualização 37
4.4.3 Resultados 37
4.4.4 Análise dos Resultados 40

5 Conclusão 43
5.1 Resultados e análises finais 43
5.2 Oportunidades de melhoria 44

Bibliografia 46

A Benchmark de Solomon 48

B Estimativa para Busca Exaustiva 51

Lista de Figuras

2.1 Diagrama de Path Relinking 14
2.2 Visão geral do Q-Learning 16
2.3 Algoritmo básico de Q-Learning 17

3.1 Distribuição da instância RC201 20
3.2 Rotas ótimas da instância RC201 21

4.1 Rotas encontradas pelas meta-heurísticas 28
4.2 Evolução da função custo pelas meta-heurísticas 29
4.3 Evolução da função custo no MAS-RL 34
4.4 Evolução da função custo no MAS-RL 38
4.5 Q-table 40
4.6 Distribuição de execução dos agentes 40

5.1 Tempo de viagem, espera e serviço 45

A.1 Mapa da instância C101 48
A.2 Mapa da instância R101 49
A.3 Mapa da instância RC101 49

Lista de Tabelas

4.1 Resultados das meta-heurísticas isoladas 29
4.2 Resultados do MAS 34
4.3 Resultados do MAS-RL 37

Capítulo 1

Introdução

Este capítulo introduz o leitor ao contexto no qual a presente pesquisa se insere, apresentando as motivações iniciais, o escopo do problema de otimização selecionado e a relevância científica da abordagem proposta. A Seção 1.1 delineia o tema central do estudo, abordando as restrições e a hierarquia do Problema de Roteamento de Veículos com Janelas de Tempo (VRPTW). Em seguida, a Seção 1.2 estabelece as delimitações do trabalho, focando no desenvolvimento e avaliação empírica de meta-heurísticas integradas em um sistema multi-agente colaborativo guiado por aprendizado por reforço. A justificativa para a aplicação destas ferramentas algorítmicas, dadas as dificuldades inerentes aos métodos exatos diante de problemas classificados como *NP-hard*, é apresentada na Seção 1.3. Por fim, as Seções 1.4, 1.5 e 1.6 expõem, respectivamente, os objetivos formulados, a metodologia adotada de melhoria contínua e a estruturação capitular do conteúdo subsequente neste documento.

1.1 Tema

O tema do trabalho é a aplicação e a análise de métodos meta-heurísticos para a resolução de problemas de otimização complexos, classificados como *NP-hard* [1], como o Problema de Roteamento de Veículos (ou *Vehicle Routing Problem* – VRP). O VRP, introduzido pioneiramente na literatura sob a ótica da distribuição de derivados de petróleo em 1959 [2], consiste em determinar o conjunto ideal de rotas para uma frota de veículos, com o objetivo de visitar um conjunto de clientes, cada

um com uma demanda específica. O problema a ser resolvido é encontrar a solução mais eficiente para este roteamento, considerando restrições como distância total, número de veículos, tempo e capacidade de entrega.

Dentro do vasto domínio do VRP, este trabalho concentra-se especificamente no Problema de Roteamento de Veículos com Janelas de Tempo (*Vehicle Routing Problem with Time Windows* – VRPTW). O VRPTW estende o problema clássico ao impor que o atendimento a cada cliente deve ser iniciado obrigatoriamente dentro de um intervalo temporal pré-definido, composto por um tempo de abertura e um tempo de fechamento (*time window*).

A inclusão destas restrições temporais torna o desafio ainda mais rigoroso, pois a viabilidade de uma rota não depende apenas da capacidade de carga do veículo, mas da coordenação precisa dos horários de chegada e dos tempos de serviço em cada nó. Neste cenário, o objetivo de encontrar a solução mais eficiente desdobra-se em uma hierarquia de otimização: busca-se, primordialmente, a minimização do número de veículos necessários e, subsequentemente, a redução da distância total percorrida, garantindo que nenhum compromisso temporal seja violado. Por ser um problema de natureza *NP-hard*, a exploração de janelas de tempo exige que os métodos meta-heurísticos aplicados possuam alta capacidade de adaptação para navegar em um espaço de busca repleto de restrições de inviabilidade.

1.2 Delimitação

O objeto de estudo são problemas de otimização *NP-hard*. O projeto se concentrará na otimização do VRPTW, que estende o clássico Problema do Caixeiro Viajante (*Traveling Salesman Problem* – TSP) para múltiplos veículos, capacidade de carga e janelas de tempo. O foco será na implementação e comparação de diferentes algoritmos meta-heurísticos (*Simulated Annealing* – SA, *Tabu Search* – TS e *Genetic Algorithm* – GA), e na integração desses métodos com um sistema multiagente (*Multi-Agent System* – MAS), conjuntamente com a aplicação de aprendizado por reforço (*Reinforcement Learning* – RL) para melhoria deste sistema.

1.3 Justificativa

O VRPTW, em virtude de sua formulação que estende o VRP clássico, é rigorosamente classificado como um problema *NP-hard* [1]. Nesse contexto de alta complexidade computacional, o esforço temporal demandado por métodos exatos para a obtenção de uma solução ótima cresce de forma exponencial em função da dimensionalidade da instância analisada, tornando tais abordagens intratáveis para a ampla escala de cenários reais. Diante dessa limitação, as meta-heurísticas despontam como uma alternativa viável, visto que, embora não assegurem a otimalidade global, oferecem mecanismos para explorar o espaço de busca eficientemente, fornecendo aproximações de alta qualidade em tempo computacional factível.

A originalidade deste trabalho reside na aplicação de um MAS com RL para integrar e aprimorar o desempenho dos diferentes métodos meta-heurísticos. O objetivo é explorar como a comunicação entre agentes pode levar a resultados superiores aos de cada algoritmo individualmente, evitando mínimos locais e alcançando uma solução globalmente mais eficiente. A importância do projeto está em sua aplicabilidade prática em áreas como logística, transporte e serviços de entrega, onde a otimização de rotas é fundamental para a eficiência e redução de custos.

Ainda no tocante à justificativa da abordagem, o Apêndice B apresenta uma análise quantitativa da grandeza temporal requerida pela força bruta para a obtenção da solução ótima nas instâncias discutidas, reiterando a necessidade de métodos heurísticos.

1.4 Objetivos

O objetivo geral é analisar e implementar um sistema híbrido, combinando métodos meta-heurísticos com um sistema multiagente e aprendizado por reforço, para otimizar a resolução do VRPTW. Como objetivos específicos, temos:

1. Formular o VRPTW, definindo a função de custo e as restrições (tempo, volume, peso) para a avaliação das soluções;

2. Implementar, de forma independente, os algoritmos meta-heurísticos: SA, TS e GA;

3. Integrar os algoritmos em um MAS, no qual a interação entre os agentes leva à colaboração na busca pela solução;

4. Aplicar RL para aprimorar a estratégia de busca dos agentes;

5. Comparar o desempenho dos algoritmos individuais com o MAS e o sistema com aprendizado por reforço (MAS-RL) para avaliar a melhoria na otimização da solução.

1.5 Metodologia

A metodologia será dividida nas seguintes etapas, focando na aplicação de uma abordagem iterativa e de melhoria contínua para a otimização do problema.

- **Modelagem do Problema:** O VRP será formulado como um problema de otimização, no qual uma solução é representada como um vetor ordenado de pontos de entrega. A função de custo avaliará a qualidade de cada solução, considerando a distância percorrida, o número de veículos e as restrições de tempo e capacidade, atribuindo pesos a cada parâmetro;

- **Implementação dos Algoritmos:** Os três algoritmos meta-heurísticos serão implementados em *Python*. Para cada algoritmo, serão definidas etapas de inicialização de uma solução aleatória, geração de novas soluções e a manutenção de um histórico para direcionar a busca;

- **Integração com Sistema Multiagente:** Os algoritmos serão encapsulados como agentes dentro de um MAS. O sistema operará de forma cooperativa, permitindo que os agentes interajam para compartilhar informações e refinar as soluções encontradas;

- **Aplicação de Aprendizado por Reforço:** O método de *Q-Learning* será aplicado para associar os estados do problema às ações (ajuste de estratégias de busca), permitindo que o sistema aprenda a selecionar a melhor abordagem de forma autônoma ao longo do tempo.

1.6 Descrição

O restante deste trabalho está organizado em quatro capítulos principais, além das referências bibliográficas e apêndices, conforme detalhado a seguir.

No Capítulo 2, é apresentada a Fundamentação Teórica, que estabelece a base conceitual necessária para a compreensão do projeto. Nele, define-se o VRPTW e detalha-se a metodologia de avaliação baseada no *Benchmark* de Solomon [3]. O capítulo descreve o funcionamento técnico do trio de meta-heurísticas selecionado e introduz a arquitetura do MAS utilizando o *framework* Mesa. Finaliza abordando as estratégias de cooperação via *Path Relinking* (PR) e a teoria de Aprendizado por Reforço.

O Capítulo 3 detalha o desenvolvimento do Método Proposto da pesquisa. Ele descreve as etapas iniciais de modelagem e as arquiteturas das classes. Em seguida, expõe-se a modelagem do sistema multiagente, demonstrando a dinâmica de cooperação por meio do *Path Relinking* e a implementação do *Q-Learning* como hiper-heurística, orquestrando e selecionando as melhores ações para esse ambiente.

Os testes realizados e a apresentação dos resultados ficam a encargo do Capítulo 4: Experimentos Computacionais. A seção avalia o desempenho prático, apresentando inicialmente o *baseline* de cada meta-heurística isolada, em sequência os resultados agregados pela cooperação simplista do sistema multiagente e, por fim, a análise do desempenho da hiper-heurística com Aprendizado por Reforço (MAS-RL).

As reflexões e fechamentos a respeito da pesquisa são apresentados no Capítulo 5, que compõe as Conclusões do trabalho. Nele, será explicitado se a integração entre MAS e RL de fato superou o desempenho dos algoritmos isolados, validando a hipótese de que a inteligência coletiva e a adaptação dinâmica de parâmetros produzam soluções superiores para o VRPTW. O capítulo encerra com a indicação de oportunidades de melhoria e sugestões para trabalhos futuros.

Capítulo 2

Fundamentação Teórica

Este capítulo apresenta a fundamentação teórica essencial para o entendimento das metodologias e soluções computacionais propostas neste trabalho. A Seção 2.1 formaliza o conceito do Problema de Roteamento de Veículos e detalha os padrões de avaliação de instâncias da literatura. A Seção 2.2 justifica a seleção das metodologias heurísticas adotadas (Recozimento Simulado, Busca Tabu e Algoritmo Genético) descrevendo seus funcionamentos e premissas. Posteriormente, a Seção 2.3 explora os princípios de um Sistema Multi-agente, abordando o *framework* de modelagem distribuída escolhido e a estratégia heurística de cooperação implementada para evitar o isolamento de regiões promissoras (*Path Relinking*). Por fim, a compreensão do Aprendizado por Reforço e a lógica da recompensa empregada na estratégia do Q-Learning, que norteia a coordenação estratégica proposta, são objetos da Seção 2.4.

2.1 Definição do problema

Para compreender a avaliação das soluções quantitativas na experimentação futura, faz-se necessário contextualizar a natureza exata e as particularidades do problema central que pauta toda a sequência teórica e empírica deste estudo. Inicialmente, a Subseção 2.1.1 define o arranjo lógico do conhecido e amplamente mapeado VRP, englobando as dificuldades computacionais e as raízes atadas à complexidade tempo de polinomial não-determinístico difícil (*NP-hard*). Secundariamente, a Subseção 2.1.2 elucida a base paramétrica de referências extraída do Benchmark de Solomon de 1987, estabelecendo um padrão ouro viável de contraste na apuração da

acurácia e otimização obtida pelas composições desenvolvidas em tempo de execução comparado.

2.1.1 O Problema de Roteamento de Veículos

O Problema de Roteamento de Veículos (VRP) é um desafio de otimização combinatória e programação linear inteira que busca determinar o conjunto ideal de rotas para uma frota de veículos entregarem mercadorias a um conjunto específico de clientes. De forma genérica, o objetivo fundamental do VRP é minimizar o custo total da operação (geralmente expresso em distância total ou tempo) garantindo que:

- Cada cliente deve ser visitado exatamente uma vez;
- Todos os veículos devem começar e terminar suas rotas em um depósito central;
- As restrições operacionais (como capacidade de carga ou janelas de tempo) devem ser estritamente respeitadas.

O VRP é classificado na ciência da computação como um problema *NP-hard* (problemas de tempo polinomial não-determinístico difícil). Isso significa que, à medida que o número de clientes aumenta, o tempo necessário para encontrar a solução ótima cresce de forma exponencial, tornando impossível o uso de métodos exatos (como a força bruta) para instâncias de larga escala.

É precisamente essa complexidade computacional inerente que fundamenta a aplicação das meta-heurísticas (SA, TS e GA) investigadas neste estudo. Em contraste com os métodos de resolução exatos, as abordagens meta-heurísticas são concebidas para encontrar soluções subótimas de alta qualidade em tempo polinomial aceitável, viabilizando assim a aplicação prática em cenários logísticos de envergadura real.

2.1.2 Metodologia de Avaliação e o Benchmark de Solomon

Para assegurar a validade científica deste estudo e permitir a comparabilidade dos resultados obtidos pelas meta-heurísticas propostas, a modelagem será conduzida estritamente sob os parâmetros do Benchmark de Solomon (1987). A utilização

de instâncias consagradas na literatura é imperativa, pois fornecem um "gabarito" técnico que permite atestar a precisão e a eficiência de cada algoritmo desenvolvido.

Dado que as instâncias de Solomon incorporam coordenadas geográficas, demandas de carga e janelas temporais, o problema aqui tratado é classificado como Problema de Roteamento de Veículos com Janelas de Tempo (VRPTW). Embora simplificada em relação às complexidades imprevisíveis do mundo real (como variações de tráfego ou clima), essa modelagem oferece um ambiente controlado e puramente matemático para a análise de desempenho algorítmico. Para que a comparação seja viável e metodologicamente aceitável, o instanciação seguirá as métricas rigorosas do *benchmark*:

1. **Função Objetivo Hierárquica:** A avaliação do desempenho seguirá uma prioridade de dois níveis:
   (a) **Minimização do número de veículos (K):** O objetivo primário é atender a todos os clientes com a menor frota possível.
   (b) **Minimização da distância total percorrida (D):** Como objetivo secundário, busca-se o percurso mais curto para a frota definida.
2. **Cálculo de Distância e Tempo:** A distância entre dois pontos (i, j) é calculada pela Distância Euclidiana plana:

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2}$$

3. Conforme as regras de Solomon, o tempo de viagem é numericamente equivalente à distância ($t_{ij} = d_{ij}$), e todos os cálculos devem utilizar precisão dupla (*double precision*), com os resultados finais de distância arredondados para duas casas decimais.
4. **Ponto de Origem e Retorno:** O cliente indexado como 0 define o depósito central, ponto obrigatório de partida e encerramento de todas as rotas.
5. **Restrições de Viabilidade:** Uma rota é considerada válida apenas se respeitar simultaneamente a Capacidade do Veículo (Q), a soma das demandas

dos clientes atendidos e o atendimento dentro das janelas de tempo estabelecidas (*Ready Time* e *Due Date*), somando-se a isso o *Service Time* (Tempo de Serviço) em cada parada.

Esta abordagem padronizada transforma o problema em um estudo de caráter matemático e exploratório, estabelecendo uma base sólida que pode, futuramente, ser adaptada para modelos logísticos de maior complexidade e variáveis dinâmicas.

Neste trabalho serão utilizadas, fundamentalmente, as instâncias de tipo RC, série 2. Ou seja, a distribuição dos clientes contém algumas aglomerações, mas também pontos aleatórios. Além disso, a capacidade dos veículos é maior e as janelas de tempo mais largas, significando muitos clientes por rota, poucos veículos necessários. Ver Apêndice A.

2.2 Seleção do Trio de Meta-heurísticas

A escolha das meta-heurísticas Recozimento Simulado, Busca Tabu e Algoritmo Genético para a resolução do VRPTW fundamenta-se na necessidade de realizar uma análise comparativa abrangente, cobrindo dois eixos de vertentes de busca em otimização combinatória:

- **Vertente de Trajetória Única vs. População**: Métodos de trajetória única trabalham com apenas uma solução por vez. No caso do VRPTW, é como se houvesse um único agente caminhando pelo mapa das rotas, tentando dar passos para direções melhores. O foco é a intensificação, lapidar uma solução até o limite. Métodos de população trabalham com múltiplas soluções simultâneas. É como se houvesse diversos agentes espalhados pelo mapa. O foco é a diversificação, explorar várias áreas ao mesmo tempo para não ficar preso em uma única região.

- **Vertente Estocástica vs. Determinística**: Esta divisão foca em como o algoritmo toma decisões. No caso estocástico, as decisões de aceitar ou não uma rota pior dependem de probabilidades. Já na vertente determinística, as decisões são baseadas em regras lógicas e no histórico.

A diversidade técnica entre esses métodos permite utilizar diferentes mecanismos de escape de ótimos locais e estratégias de exploração do espaço de busca diante das restrições de roteamento. Busca-se usar suas maiores forças em conjunto: aleatoriedade, memória e compartilhamento de informações entre soluções. A seguir, cada método tem sua escolha justificada em mais detalhes.

2.2.1 Recozimento Simulado

[4] Método de trajetória única e estocástico. Sua inclusão é justificada pela sua capacidade de exploração global por meio de um mecanismo probabilístico. Se Δf é a diferença de custo entre a nova solução (s') e a atual (s):

$$\Delta f = f(s') - f(s)$$

A probabilidade P de aceitação é definida por:

$$P(\text{aceitar } s') = \begin{cases} 1, & \text{se } \Delta f < 0 \\ e^{-\frac{\Delta f}{T}}, & \text{se } \Delta f \geq 0 \end{cases}$$

Onde T é a temperatura, que decai ao longo do tempo conforme uma taxa de resfriamento α:

$$T_{k+1} = \alpha \cdot T_k$$

No contexto do VRPTW, o SA é essencial para demonstrar como a aceitação controlada de soluções de pior qualidade, governada por um parâmetro de temperatura, permite que o algoritmo escape de ótimos locais prematuros.

2.2.2 Busca Tabu

[5] Método de trajetória única e determinístico. A justificativa reside no seu uso de "listas tabu", que impedem o algoritmo de retornar a soluções visitadas recentemente, forçando uma exploração sistemática e agressiva de novas vizinhanças. Mais objetivamente, o algoritmo opera sobre a vizinhança $N(s)$ da solução atual, mas subtrai os movimentos que estão na Lista Tabu (L):

$$s_{next} = \text{argmin}\{f(s') \mid s' \in N(s) \setminus L\}$$

Entretanto, existe o Critério de Aspiração. Se uma solução s' está na lista tabu, mas seu custo é melhor que a melhor solução encontrada até agora (s_{best}), o estado tabu é ignorado:

$$\text{Se } f(s') < f(s_{best}), \text{ então } s' \in \mathit{Candidatos} \text{ mesmo se } s' \in L$$

Isso mostra que a memória do algoritmo é inteligente e não impede o progresso caso uma solução excepcional seja encontrada por um caminho anteriormente proibido.

Além disso, métodos que residem suas maiores forças no uso de memória são amplamente reconhecidos na literatura como técnicas eficazes para problemas de logística (como o VRPTW).

2.2.3 Algoritmo Genético

[6, 7] Método de população e estocástico. A escolha do GA justifica-se pela sua habilidade de diversificação. Por meio dos operadores de cruzamento e mutação, o algoritmo é capaz de recombinar sub-rotas eficientes de diferentes soluções para gerar descendentes potencialmente superiores. Isso se traduz em converter o problema de minimização de custos em um valor de sobrevivência, representado pela função de aptidão (*fitness*) $F(s)$:

$$F(s) = \frac{1}{C(s) + \epsilon} \tag{2.1}$$

Onde ϵ é um valor infinitesimal para evitar divisão por zero. Presente, neste caso, apenas por rigor teórico, pois a função de custo que será utilizada é sempre maior que 0. A probabilidade de um indivíduo ser selecionado para o cruzamento é dada por:

$$P_i = \frac{F(s_i)}{\sum_{j=1}^{N} F(s_j)} \tag{2.2}$$

Onde N é o tamanho da população. Isso garante que rotas mais curtas tenham maior probabilidade biológica de transmitir seus genes (sequências de clientes).

No VRPTW, isso permite observar como características geográficas de boas rotas podem ser preservadas e transmitidas, oferecendo uma dinâmica de busca completamente distinta dos métodos de trajetória única.

2.3 Sistema multi-agente

Os *Multi-Agent Systems* (MAS) consistem em um conjunto de elementos computacionais, denominados agentes, que interagem em um ambiente compartilhado para resolver problemas que ultrapassam as capacidades individuais de um único processador ou algoritmo [8]. Tais sistemas têm sido aplicados com sucesso em diversas áreas, incluindo otimização de colônias de formigas, onde a emergência de comportamentos complexos globais decorre das interações locais entre agentes relativamente simples.

No contexto do VRPTW, a abordagem multiagente é empregada para decompor a complexidade da busca em diferentes especialidades: enquanto um agente foca na exploração global do espaço de busca (*exploration*), outro dedica-se ao refinamento local intensivo (*exploitation*).

Essa arquitetura permite a transição de algoritmos competitivos para um modelo cooperativo, onde a inteligência coletiva emerge da troca de informações entre os agentes. A principal vantagem desta abordagem para o VRPTW é a capacidade de evitar que o sistema fique preso em ótimos locais, uma vez que diferentes comportamentos heurísticos operam simultaneamente e compartilham suas melhores descobertas por meio de uma memória comum.

2.3.1 Framework Mesa

O Mesa é uma biblioteca de código aberto desenvolvida em *Python* para a modelagem baseada em agentes, permitindo que pesquisadores a criação, manipulação e análise de sistemas complexos compostos por múltiplos elementos autônomos [9].

Diferente de modelos matemáticos puramente estáticos, o Mesa fornece uma estrutura computacional que facilita a observação de comportamentos emergentes a partir das interações locais entre agentes em um ambiente dinâmico.

A arquitetura do Mesa é fundamentada em uma separação clara de responsabilidades, baseada em três componentes principais que seguem padrões de projeto modernos:

- **Agente (*Agent*):** Representa a unidade fundamental de inteligência e ação. Cada agente possui seu próprio estado e lógica interna de decisão, sendo capaz de interagir com o ambiente ou com outros agentes.
- **Modelo (*Model*):** Atua como a entidade de alto nível que orquestra o sistema. O modelo define as regras do ambiente, mantém os dados globais e gerencia a criação e destruição dos agentes.
- **Agendador (*Scheduler*):** É o componente responsável pela coordenação temporal das ações. Ele determina a ordem em que os agentes executam seus passos (*steps*), podendo operar de forma aleatória, sequencial ou simultânea, o que impacta diretamente a dinâmica de convergência do sistema.

No âmbito da otimização combinatória, o Mesa destaca-se por permitir que meta-heurísticas distintas sejam encapsuladas em agentes independentes, transformando um problema de busca individual em um processo de cooperação distribuída.

2.3.2 Estratégia de cooperação: Path Relinking

O PR é uma estratégia de intensificação de busca originalmente proposta como um componente de memória para a Busca Tabu [10], mas que se consolidou como uma técnica poderosa para a integração de soluções em diversos contextos de otimização combinatória.

O conceito fundamental do *PR* reside na exploração sistemática de trajetórias que conectam duas soluções de alta qualidade, denominadas solução inicial (S_i) e solução guia (S_g). A premissa teórica do método é que, ao navegar pelo caminho que transforma a solução inicial na solução guia, o algoritmo pode descobrir soluções

intermediárias que compartilham características de ambas e que, em muitos casos, superam as referências originais em termos de qualidade.

Esse processo é realizado por meio de uma sequência de movimentos de vizinhança que reduzem progressivamente a distância, comumente medida pela distância de Hamming ou por diferenças estruturais em rotas, entre as duas soluções.

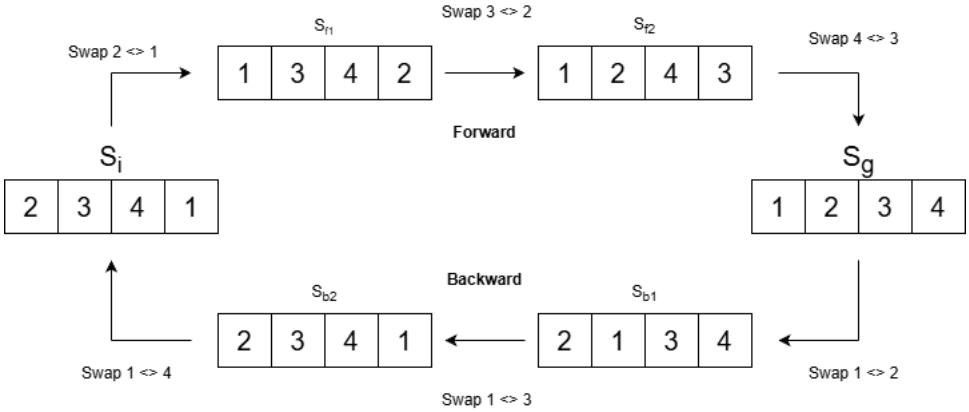

Figura 2.1: Representação do caminho entre S_i e S_g.

No contexto do *VRPTW*, o *Path Relinking* atua como um mecanismo de cooperação de alto nível ao permitir a sinergia estrutural, combinando a eficiência de sequenciamento de uma solução com a estrutura de particionamento de veículos de outra para criar rotas que nenhum agente isolado seria capaz de conceber. Além disso, a técnica possibilita a exploração de vizinhanças distantes, uma vez que, ao contrário de buscas locais tradicionais que exploram apenas o entorno imediato, o PR investiga uma região específica do espaço de busca delimitada por duas soluções de elite, focando o esforço computacional em áreas comprovadamente promissoras.

Outro fator determinante é a sua capacidade de facilitar o escape de ótimos locais, pois a trajetória entre as soluções pode atravessar vales de soluções inviáveis ou de baixa qualidade para alcançar picos de aptidão mais elevados, agindo como um catalisador para a convergência global do sistema multiagente. Dessa forma, o PR não é apenas um método de busca, mas uma filosofia de cooperação que utiliza o conhecimento acumulado por diferentes agentes especialistas para sintetizar soluções superiores, mitigando as fraquezas individuais observadas em execuções isoladas.

Como seus operadores e manipuladores de soluções são relativamente simples, o PR é menos custoso computacionalmente que uma meta-heurística, seu esforço reside majoritariamente na verificação das funções de custo. Este é seu ponto forte, mas também ponto fraco, uma vez que depende de boas soluções como entrada para melhorá-las ainda mais. É evidente, portando que deve ser usada em combinação com outros algoritmos, idealmente ao final das etapas.

2.4 Aprendizado por Reforço

O RL é uma área do aprendizado de máquina que se preocupa com a forma como agentes inteligentes devem tomar ações em um ambiente a fim de maximizar uma recompensa acumulada. Diferente do aprendizado supervisionado, que se baseia em um conjunto de dados rotulados, fundamenta-se na experiência direta do agente, que aprende por meio de um processo de tentativa e erro ao interagir com o ambiente [11].

A estrutura básica de um problema de RL é frequentemente modelada como um Processo de Decisão de Markov, que envolve os seguintes componentes fundamentais:

- **Agente (*Agent*):** O tomador de decisão que observa o ambiente e executa ações.
- **Ambiente (*Environment*):** O contexto externo com o qual o agente interage.
- **Estado (*State*):** Uma representação da situação atual do ambiente em um determinado instante. No contexto de otimização, o estado pode refletir a qualidade da solução atual, como o número de veículos e a distância total.
- **Ação (*Action*):** O conjunto de decisões possíveis que o agente pode tomar.
- **Recompensa (*Reward*):** Um sinal numérico de *feedback* enviado pelo ambiente após uma ação, que indica o sucesso ou fracasso daquela decisão em relação ao objetivo final.

O objetivo do RL é encontrar uma política (π), que é um mapeamento de estados para ações que maximizem o retorno esperado a longo prazo. Um conceito crítico neste processo é o equilíbrio entre exploração (*exploration*), onde o agente testa novas ações para descobrir sua utilidade, e explotação (*exploitation*), onde o agente utiliza o conhecimento já adquirido para escolher as ações que renderam as maiores recompensas anteriormente.

2.4.1 Q-Learning

O Q-Learning [12] é um algoritmo de Aprendizado por Reforço do tipo *off-policy* e *model-free*, o que significa que ele pode aprender a política ótima de tomada de decisão sem a necessidade de um modelo prévio das transições do ambiente ou das probabilidades de recompensa. O "Q" na nomenclatura refere-se à Qualidade (ou valor de utilidade), representando o quão proveitosa é uma determinada ação em um estado específico.

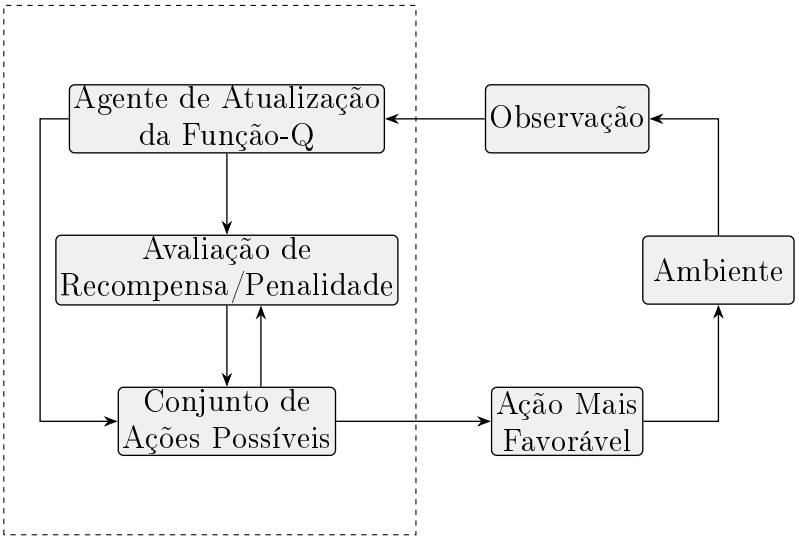

Figura 2.2: Esquema de interação entre agente e ambiente no processo de aprendizagem.

O núcleo do algoritmo consiste em estimar os valores da função $Q(s, a)$, que mensura a recompensa total esperada a longo prazo que um agente pode obter ao realizar a ação a partindo do estado s. Em problemas com espaços de estados discretos, esses valores são armazenados em uma estrutura de dados conhecida como Tabela-Q (*Q-Table*), onde as linhas representam os estados do sistema e as colunas as ações possíveis.

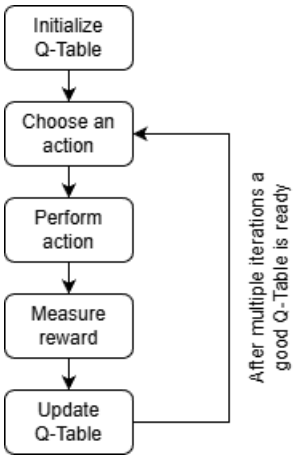

Figura 2.3: Diagrama do algoritmo de atualização do valor-Q.

O aprendizado ocorre através da atualização iterativa dos valores-Q (*Q-values*) conforme o agente explora o ambiente. Essa atualização fundamenta-se na Equação de Bellman, que ajusta o valor atual com base na recompensa imediata recebida e na melhor estimativa de recompensas futuras:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)] \tag{2.3}$$

Nesta formulação, os parâmetros de controle desempenham papéis cruciais:

- **Taxa de Aprendizado (α):** Determina o quanto a nova informação substitui o conhecimento antigo.
- **Recompensa Imediata (r):** É o *feedback* numérico direto recebido do ambiente após a execução de uma ação. Representa o ganho instantâneo e serve como a base factual para atualizar a estimativa de qualidade daquela ação.
- **Fator de Desconto (γ):** Define a importância das recompensas futuras em relação às imediatas.
- **Diferença Temporal:** O termo entre colchetes representa o erro de predição, ou seja, a diferença entre o valor que o agente esperava e o que ele efetivamente percebeu após a transição para o novo estado s'.

Para que o algoritmo seja eficaz, o agente deve balancear a busca por novos conhecimentos com o uso do conhecimento já adquirido. A estratégia mais comum é a $\epsilon - greedy$, onde com uma probabilidade ϵ o agente escolhe uma ação aleatória (exploração) e com probabilidade $1-\epsilon$ ele seleciona a ação com o maior valor-Q atual para aquele estado (explotação). Ao final do processo de treinamento, a Tabela-Q converge para uma representação da estratégia ótima, permitindo que o sistema, ao se deparar com uma determinada configuração do problema, saiba as melhores ações a serem tomadas.

Capítulo 3

Método Proposto

Este capítulo descreve a materialização metodológica da pesquisa teórica, mapeando as fases de delineamento das implementações executadas nos estudos de caso. A primeira estruturação analítica está descrita na Seção 3.1 com pautas iniciais como pré-processamento de datasets cartesianos, extração dos parâmetros georreferenciados para as estruturas na modelagem computacional e a verificação empírica das rotinas em base conhecida. Em progressão, a Seção 3.2 introduz e apresenta esquematicamente o ambiente multi-agente estabelecido através dos componentes unificados baseados no *framework Mesa*, elucidando o comportamento reativo da seleção cooperativa das soluções e da unificação interativa pelo intermédio da exploração bidirecional por intermédio do *Path Relinking*. A maturação e avanço do paradigma do sistema inteligente colaborativo culmina na Seção 3.3 com o controle da centralidade coordenadora repassada para as bases do Q-Learning, estruturada com recompensas dinâmicas, decaimento de aleatoriedade perante convergência nas iterações e gestão estocástica adaptativa.

3.1 Etapas iniciais

As fases preliminares do desenvolvimento focaram na construção da arquitetura base do sistema. Foram implementadas as rotinas de entrada e saída de dados, responsáveis pela leitura dos *datasets* e dos arquivos de solução de referência (*benchmarks*), além de módulos para a visualização gráfica de rotas e monitoramento de desempenho.

Um componente crítico nesta etapa foi a função custo compartilhada, desenvolvida de forma modular para garantir que a avaliação das rotas fosse consistente entre os diferentes algoritmos testados. Para validar a implementação, utilizou-se como referência a **instância RC201** de Solomon (resultado e procedimentos seguidos para alcançar tal, respectivamente em [13, 14], que também foi a base para todos os experimentos deste trabalho. A figura abaixo apresenta a distribuição espacial dos clientes nesta instância.

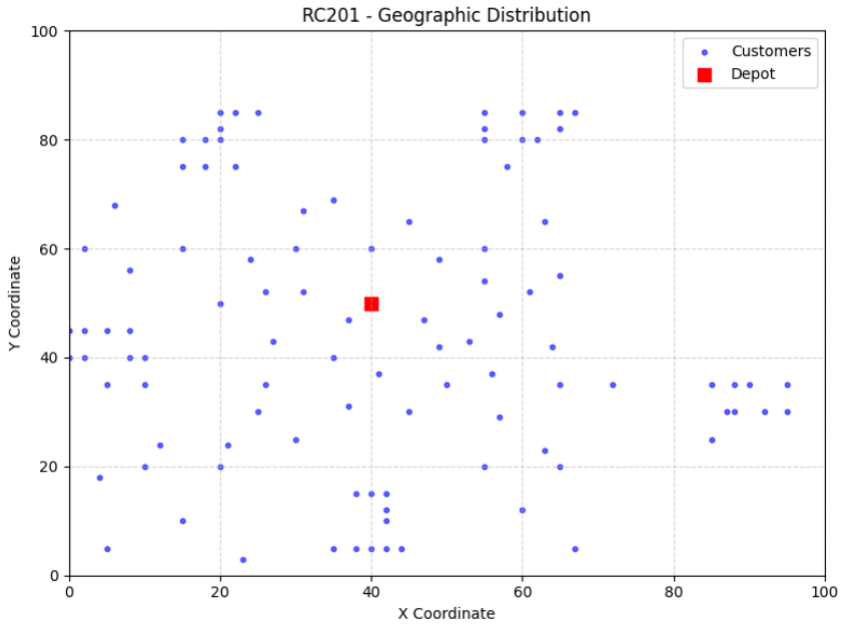

Figura 3.1: Distribuição espacial dos clientes e do depósito na instância RC201 de Solomon.

A validação da função custo ocorreu por meio da leitura da melhor solução conhecida para a RC201. Ao processar essa solução e comparar os resultados obtidos com os indicadores do *benchmark*, confirmou-se a integridade do cálculo de custo. A figura abaixo ilustra o conjunto de rotas ótimas para este cenário.

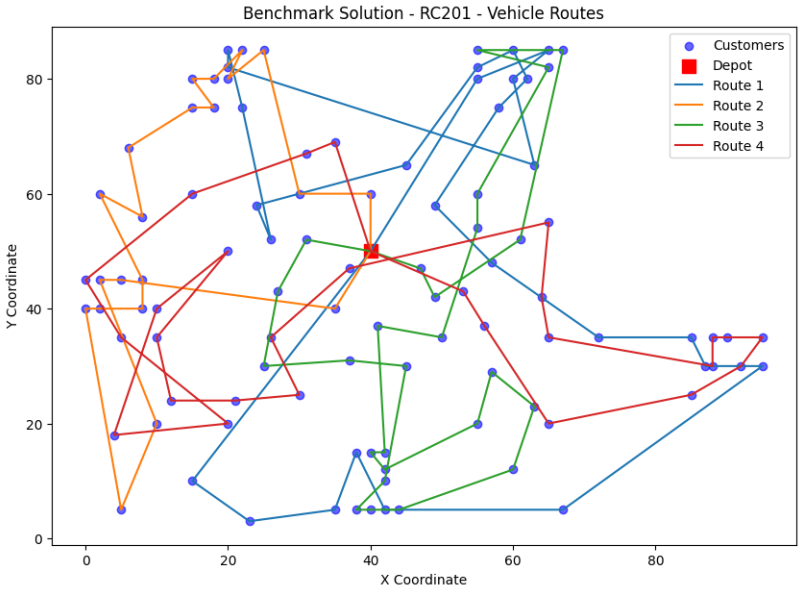

Figura 3.2: Visualização do conjunto de subrotas da melhor solução conhecida para a instância RC201.

Por fim, foram estruturadas as classes base para os algoritmos meta-heurísticos. Essa arquitetura orientada a objetos permitiu a execução dos testes preliminares e a coleta dos primeiros dados para análise comparativa.

Um ponto importante a se notar é a escolha de não separar rotas em viável ou inviável ao longo dos algoritmos. Todas as rotas foram consideradas realizáveis, de forma que a inserção de retorno ao depósito nas rotas se dá pela verificação da impossibilidade de seguir para um próximo cliente, seja pela limitação de carga ou temporal.

3.2 Implementação de sistema Multi-Agente

A arquitetura do sistema proposto foi desenvolvida utilizando o *framework Mesa*, que permite a modelagem de agentes autônomos em um ambiente controlado. No contexto deste trabalho, o MAS atua como uma camada de coordenação que integra as meta-heurísticas independentes, transformando-as em agentes especialistas que colaboram para a otimização do VRPTW.

3.2.1 Arquitetura e Componentes do Sistema

O modelo é estruturado em três componentes principais, conforme implementado na classe `VRPOptimizationModel`:

- **Agentes Especialistas (*HeuristicAgent*):** Cada instância representa uma meta-heurística (SA, TS ou GA) que atua de forma independente. Em cada ciclo de execução (*step*), o agente recupera uma solução promissora do *pool* de elite para servir como uma solução inicial (evidentemente, em estado melhor que uma solução inicial puramente aleatória), executa sua lógica de busca e devolve o resultado aprimorado ao sistema.

- **Memória Compartilhada (*ElitePool*):** Funciona como um quadro negro (*blackboard*) que armazena as melhores soluções encontradas por todos os agentes. O *pool* possui um tamanho fixo, garantindo que apenas as soluções com menor custo (considerando o número de veículos K e distância total D) sejam mantidas e compartilhadas.

- **Mecanismo de Coordenação e Sincronização:** O modelo gerencia a execução dos agentes de forma embaralhada (*shuffle_do*), garantindo que a ordem de atuação não vicie a busca e que todos tenham acesso equitativo aos recursos de memória.

3.2.2 Cooperação e a Estratégia de Path Relinking

A verdadeira cooperação no SMA ocorre através do método `run_path_relinking`. Diferente de uma execução paralela simples, este sistema implementa uma busca cooperativa onde duas soluções de elite são combinadas:

1. **Seleção por Diversidade:** O sistema seleciona a melhor solução atual (*source*) e a compara com outros candidatos no *ElitePool*, escolhendo aquele com maior distância de *Hamming* (maior diversidade) como alvo (*target*).

2. **Exploração Bidirecional:** Adotou-se a estratégia de *Path Relinking* bidirecional, na qual a exploração ocorre simultaneamente em dois sentidos: da solução *source* em direção à *target* e vice-versa. Essa abordagem dobra as trajetórias exploradas entre os dois pontos de elite, permitindo que o algoritmo

investigue vizinhanças distintas que poderiam ser negligenciadas em uma busca unidirecional.

3. **Permutação e Reavaliação:** Durante a trajetória, a técnica gera caminhos intermediários permutando clientes e reavaliando a função de custo a cada passo. O uso do critério bidirecional acelera o encontro de soluções que compartilham características de ambos os genitores de elite.

4. **Sintetização de Soluções:** Se uma solução intermediária superar as originais em termos de custo (veículos K ou distância D), ela é adicionada ao *pool*, permitindo que o sistema escape de ótimos locais ao fundir características de rotas distintas geradas por agentes diferentes.

3.3 Hiper-heurística baseada em Aprendizado por Reforço

A evolução do sistema multiagente convencional para o modelo proposto reside na implementação da classe `VRPOptimizationModelRL`. Neste modelo, o mecanismo de coordenação deixa de ser puramente aleatório e passa a utilizar o algoritmo *Q-Learning* para orquestrar a ativação dos agentes especialistas, configurando uma abordagem de hiper-heurística baseada em aprendizado [15].

3.3.1 Modelagem do Q-Learning

O agente de RL opera sobre um ciclo de percepção e ação, onde a política de decisão é refinada com base na experiência acumulada no ambiente de busca. Os componentes do modelo são definidos como segue:

- **Espaço de Estados (S):** O estado é definido pela tupla discreta (K, D_{bin}). Para evitar a explosão do espaço de estados e garantir a convergência da Tabela-Q, a distância total (D) é discretizada em faixas (*bins*) de 100 unidades através do parâmetro `distance_bin_size`.

- **Espaço de Ações (A):** Corresponde à seleção de uma das três meta-heurísticas

disponíveis para execução no ciclo atual: *Simulated Annealing* (SA), *Tabu Search* (TS) ou *Genetic Algorithm* (GA).

- **Função de Recompensa Hierárquica (R):** A recompensa é estruturada para refletir a prioridade multiobjetivo do $VRPTW$:
  - **Redução de Frota:** Atribui-se uma recompensa massiva (+200) caso a ação resulte na diminuição do número de veículos (K).
  - **Redução de Distância:** Caso K se mantenha constante, a recompensa é equivalente à magnitude da melhoria em D.
  - **Penalidade por Estagnação:** Aplica-se uma pequena penalidade (-1) se a ação não gerar melhorias, incentivando a exploração de outros agentes.
- **Política de Seleção:** Utiliza-se a estratégia ϵ-*greedy* com $\epsilon = 0,4$, equilibrando a explotação do conhecimento atual com a exploração de novas trajetórias de busca.

3.3.2 Dinâmica de Cooperação e Atualização

A cada *step* do modelo, o sistema realiza três iterações de escolha e execução. Após a execução do agente selecionado, o valor de utilidade $Q(s, a)$ é atualizado na Tabela-Q utilizando a Equação 2.3:

Os hiperparâmetros adotados incluem uma taxa de aprendizado $\alpha = 0,2$ e um fator de desconto $\gamma = 0,8$, priorizando a importância de recompensas a curto e médio prazo no processo de otimização. Paralelamente ao RL, o método `run_path_relinking` continua a promover a síntese de soluções de elite, agora alimentado por um *pool* cujos candidatos são gerados por agentes selecionados de forma adaptativa.

Capítulo 4

Experimentos Computacionais

O intuito delineado para este capítulo é a observação fundamentada e relatoria exaustiva sobre as dinâmicas de comportamento captadas durante a maratona dos cálculos propostos da heurística gerada versus o cenário preestabelecido para testes e referências. Dessa forma, a Seção 4.1 garante a transparência detalhando as configurações de hardware, softwares base e bibliotecas com os arranjos empíricos fixados do tempo máximo de CPU para cada bateria experimental. Dando andamento de forma segmentada, a Seção 4.2 expõe e analisa os rendimentos brutos unitários dos métodos Recozimento Simulado, Busca Tabu e Algoritmo Genético de maneira segregada. Subsequentemente contrapõe-se o contexto anterior com a arquitetura colaborativa Multi-Agente Simples submetida às mesmas bases analíticas na Seção 4.3, evidenciando as mudanças nos índices de redução global da frota exigida ao fim cronológico. Rematando essa gradação, a Seção 4.4 afere o pico do arranjo integratizado ao dispor os relatórios perante o painel da hiper-heurística gerenciada por aprendizado, quantificando o custo marginal computacional no balanceamento tático da aprendizagem com explotação iterativa do estado-ação na meta primária.

4.1 Software e Hardware

Uma vez que o tempo de processamento é relevante para a análise deste trabalho, serão especificadas as configurações do ambiente nos quais os testes foram realizados, bem como o código fonte utilizado, permitindo a reprodutibilidade dos resultados.

```
Sistema Operacional: Windows 11 Home 64 bits (10.0, build 22631)
```

```
Fabricante do Sistema: Dell Inc.
Modelo do Sistema: Inspiron 14 5435
BIOS: 1.17.0
Processador: AMD Ryzen 7 7730U com Radeon Graphics (16 CPUs) - 2.0GHz
Memoria: 16384MB RAM (16GB)
```

O trabalho foi desenvolvido com `Python v3.11.9`, cujos principais módulos utilizados, bem como suas versões, estão listados abaixo:

```
matplotlib              3.10.6
Mesa                    3.3.1
numpy                   2.3.3
pandas                  2.3.2
```

O código fonte se encontra em: https://github.com/hugodba/TCC

Dado que os algoritmos podem continuar executando por tempo indefinido, o critério de parada foi balizado pelo tempo de execução, buscando-se por volta de ao menos 300 segundos no ambiente de *hardware* utilizado. Quanto à instrumentação dos testes, adotou-se, para mensuração do tempo, a função `time.process_time()`, que contabiliza exclusivamente o tempo de CPU dedicado ao processo, ignorando períodos de ociosidade (*sleep*) e operações de entrada/saída (I/O). Essa escolha garante que a métrica de desempenho reflita mais fielmente o esforço computacional do algoritmo.

Para reprodutibilidade, com o intuito de mitigar a aleatoriedade inerente às metaheurísticas, foi implementado o uso de *seeds* fixas nos geradores de números aleatórios. Tal procedimento assegura que os experimentos sejam determinísticos, permitindo a comparação direta entre diferentes rodadas e configurações. Além disso, para obtenção de maior validade estatística, cada algoritmo foi executado 10 vezes e foi levado em consideração suas médias de resultado nas análises e comparações.

4.2 Testes com metaheurísticos puros

Inicialmente, os três algoritmos foram avaliados de forma isolada, visando aferir suas performances e características intrínsecas, tanto em termos gerais quanto nas

especificidades do $VRPTW$. Essa análise individual é fundamental para compreender como cada meta-heurística pode contribuir para uma solução híbrida, permitindo que as forças de um método compensem as limitações do outro.

A calibração dos hiperparâmetros foi realizada predominantemente de forma empírica. Contudo, zelou-se pela proporcionalidade entre as variáveis, mantendo a coerência com os valores consolidados na literatura técnica.

4.2.1 Resultados

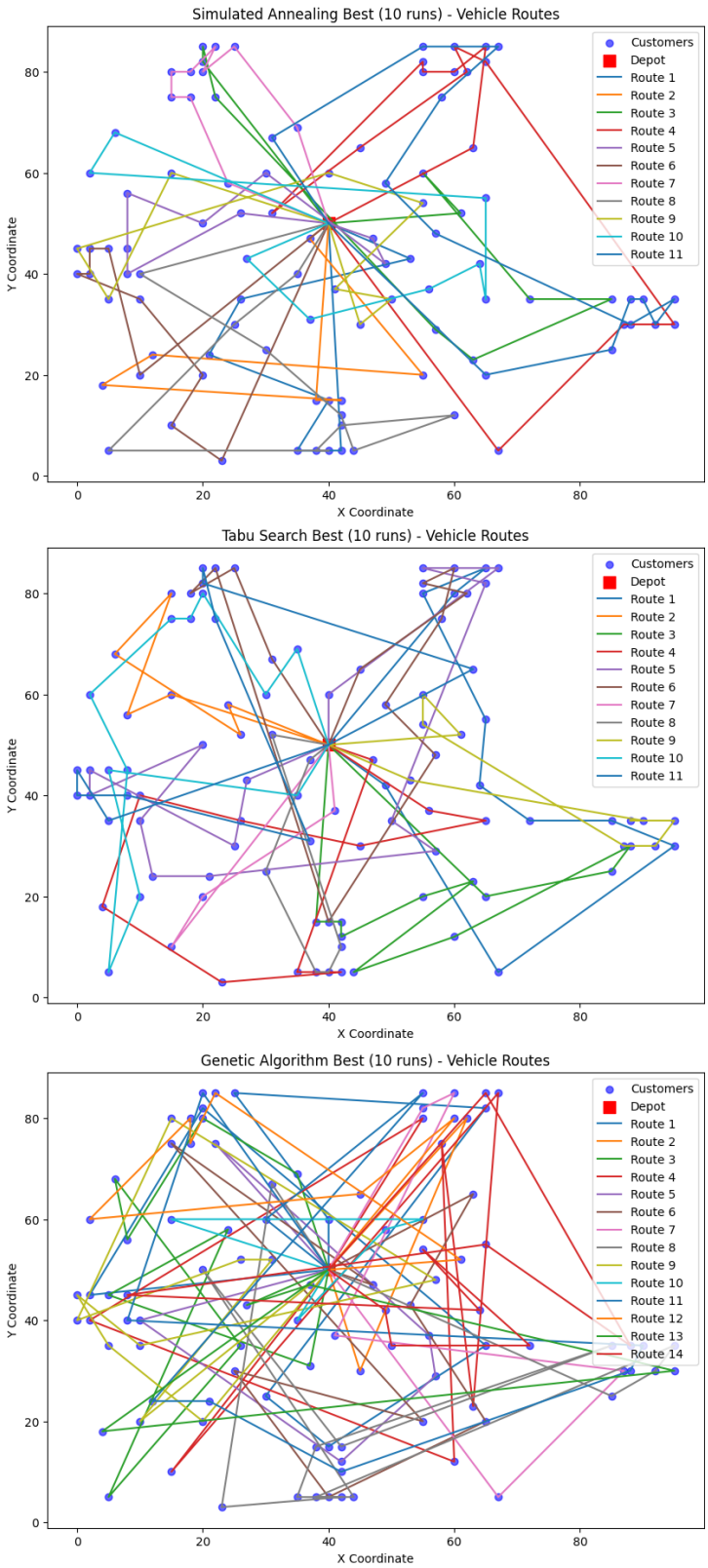

Figura 4.1: Visualização dos melhores conjuntos de subrotas das soluções encontradas pelas meta-heurísticas

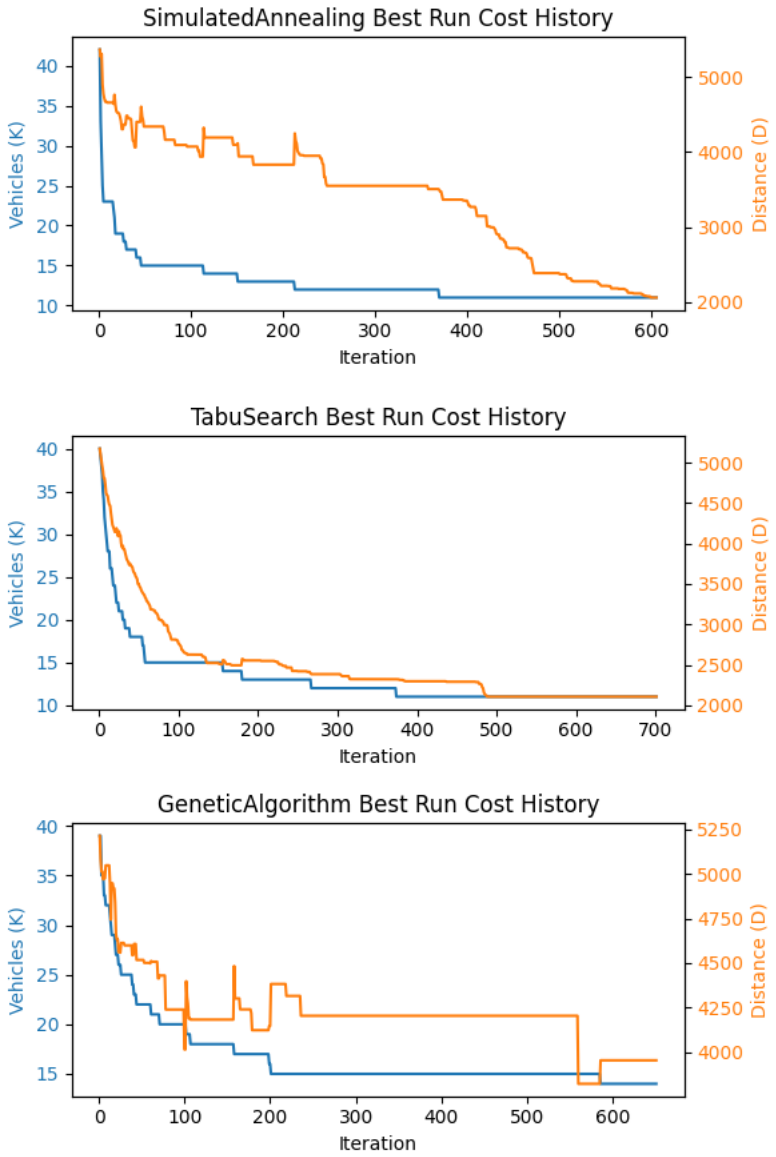

Figura 4.2: Evolução da função custo para as melhores soluções encontradas por cada meta-heurística

Tabela 4.1: Resultados dos algoritmos avaliados de forma isolada na instância RC201. Note que a coluna *iterações* representa a quantidade de avaliações da função custo, ou seja, os eixos X dos gráficos da Figura 4.2

Algoritmo	**Tempo Médio (s)**	**Iterações**	**Média K**	**Média D**	**Melhor Solução**
Simulated Annealing	$310,34 \pm 8,75$	606	$11,50 \pm 0,53$	$2151,30 \pm 58,61$	$K = 11, D = 2062,88$
Tabu Search	$309,41 \pm 11,77$	701	$12,00 \pm 0,82$	$2195,63 \pm 125,05$	$K = 11, D = 2106,50$
Genetic Algorithm	$356,83 \pm 6,29$	651	$14,90 \pm 0,74$	$4020,04 \pm 245,83$	$K = 14, D = 3952,73$

4.2.2 Análise comparativa

Os resultados da Tabela 4.1 indicam que as meta-heurísticas de busca local (SA e TS) superaram significativamente a abordagem populacional (GA) para este problema específico.

O SA apresentou a melhor performance geral, atingindo a menor distância média ($D = 2151,30$) e a melhor solução individual ($K = 11, D = 2062,88$). O TS demonstrou resultados muito próximos ao SA, com uma média de veículos de $K = 12,00$. É notável que, apesar de uma distância média ligeiramente superior, seu tempo de execução foi o menor entre os três ($309,41s$). Já o GA mostrou-se menos eficiente para a instância RC201, com médias de K ($14,90$) e D ($4020,04$) consideravelmente mais altas, além de configurar-se como o algoritmo com maior demanda computacional ($356,83s$).

Observando o comportamento de convergência na Figura 4.2, notam-se perfis distintos de evolução. O SA exibe uma redução acentuada do número de veículos no início, enquanto a redução da distância apresenta um comportamento mais linear. Ao reconfigurar as sub-rotas para priorizar a redução da frota — objetivo principal do problema — o algoritmo frequentemente sacrificou o objetivo secundário, gerando pioras temporárias na distância D. Nas iterações finais, as oscilações diminuem drasticamente devido ao efeito do resfriamento da temperatura, momento em que o critério de aceitação torna-se mais rigoroso e o algoritmo foca exclusivamente no ajuste fino (*fine-tuning*) da solução.

O TS mostra uma queda abrupta nas iterações iniciais tanto para K quanto para D, estabilizando-se rapidamente sem apresentar picos de piora na distância. Isso evidencia uma alta capacidade de refino de soluções, mas também uma relativa inabilidade em explorar novas regiões do espaço de busca (*exploration*). Tal característica provavelmente resultou na estagnação do algoritmo ao atingir as vizinhanças de um ótimo local.

Já o gráfico do GA revela uma dinâmica de busca populacional que enfrenta desafios significativos de estabilidade diante das restrições do VRPTW. A curva de

distância exibe um comportamento extremamente serrilhado, sugerindo que os operadores de *crossover* e mutação geram descendentes que frequentemente sacrificam a eficiência geográfica. Embora tenha reduzido a frota inicialmente, o algoritmo atingiu um platô de estagnação por volta da iteração 200. Este fenômeno sinaliza uma convergência prematura, onde a população tornou-se geneticamente similar, exaurindo a capacidade de descoberta de melhorias.

A análise cruzada estabelece que o GA, embora potente em diversificação, mostrou-se ineficiente comparado aos demais métodos, confirmando que algoritmos de trajetória única (SA e TS) tendem a apresentar resultados mais imediatos e consistentes para as especificidades da instância RC201.

Número de iterações e tempo de execução

Como observado na Tabela 4.1, o algoritmo *Tabu Search* (TS) realizou o maior número de iterações (701) no menor tempo médio observado ($309,41s$). Em contrapartida, o *Genetic Algorithm* (GA) executou significativamente menos iterações (651) em um intervalo de tempo muito superior ($356,83s$). Esta discrepância evidencia que o tempo de execução é influenciado por outros recursos computacionais além do simples ciclo de avaliação da função custo.

No caso do GA, o esforço adicional advém do gerenciamento de uma população de soluções. Operações como a seleção de indivíduos, o cruzamento (*crossover*) e a mutação exigem manipulação intensiva de memória e estruturas de dados complexas a cada geração, o que encarece o custo computacional de cada iteração isolada. Já o TS, embora realize mais iterações, foca em uma trajetória única de busca local, onde o custo predominante reside na exploração da vizinhança e na manutenção da lista tabu, operações que podem ser mais leves em termos de processamento por iteração do que a evolução de uma população inteira.

O *Simulated Annealing* (SA) apresenta um comportamento intermediário, com 606 iterações em $310,34s$. Nele, o tempo é consumido principalmente pelo sorteio probabilístico e pelo resfriamento da temperatura, fatores que, somados à comple-

xidade da instância RC201, resultam em uma cadência de iterações distinta dos demais métodos.

Essa falta de correlação direta entre o número de iterações e o tempo total reforça a necessidade de utilizar métricas como o `time.process_time()` para isolar o esforço de CPU, garantindo que a análise reflita a complexidade real de cada implementação frente ao problema de rotas.

4.3 Implementação de sistema Multi-Agente

A arquitetura do sistema proposto foi desenvolvida utilizando o *framework Mesa*, que permite a modelagem de agentes autônomos em um ambiente controlado. No contexto deste trabalho, o MAS atua como uma camada de coordenação que integra as meta-heurísticas independentes, transformando-as em agentes especialistas que colaboram para a otimização do VRPTW.

4.3.1 Arquitetura e Componentes do Sistema

O modelo é estruturado em três componentes principais, conforme implementado na classe `VRPOptimizationModel`:

- **Agentes Especialistas (*HeuristicAgent*):** Cada instância representa uma meta-heurística (SA, TS ou GA) que atua de forma independente. Em cada ciclo de execução (*step*), o agente recupera uma solução promissora do *pool* de elite para servir como uma solução inicial (evidentemente, em estado melhor que uma solução inicial puramente aleatória), executa sua lógica de busca e devolve o resultado aprimorado ao sistema.

- **Memória Compartilhada (*ElitePool*):** Funciona como um quadro negro (*blackboard*) que armazena as melhores soluções encontradas por todos os agentes. O *pool* possui um tamanho fixo, garantindo que apenas as soluções com menor custo (considerando o número de veículos K e distância total D) sejam mantidas e compartilhadas.

- **Mecanismo de Coordenação e Sincronização:** O modelo gerencia a execução dos agentes de forma embaralhada (*shuffle_do*), garantindo que a ordem de atuação não vicie a busca e que todos tenham acesso equitativo aos recursos de memória.

4.3.2 Cooperação e a Estratégia de Path Relinking

A verdadeira cooperação no SMA ocorre através do método `run_path_relinking`. Diferente de uma execução paralela simples, este sistema implementa uma busca cooperativa onde duas soluções de elite são combinadas:

1. **Seleção por Diversidade:** O sistema seleciona a melhor solução atual (*source*) e a compara com outros candidatos no *ElitePool*, escolhendo aquele com maior distância de *Hamming* (maior diversidade) como alvo (*target*).

2. **Exploração Bidirecional:** Adotou-se a estratégia de *Path Relinking* bidirecional, na qual a exploração ocorre simultaneamente em dois sentidos: da solução *source* em direção à *target* e vice-versa. Essa abordagem dobra as trajetórias exploradas entre os dois pontos de elite, permitindo que o algoritmo investigue vizinhanças distintas que poderiam ser negligenciadas em uma busca unidirecional.

3. **Permutação e Reavaliação:** Durante a trajetória, a técnica gera caminhos intermediários permutando clientes e reavaliando a função de custo a cada passo. O uso do critério bidirecional acelera o encontro de soluções que compartilham características de ambos os genitores de elite.

4. **Sintetização de Soluções:** Se uma solução intermediária superar as originais em termos de custo (veículos K ou distância D), ela é adicionada ao *pool*, permitindo que o sistema escape de ótimos locais ao fundir características de rotas distintas geradas por agentes diferentes.

4.3.3 Resultados

Assim como para as meta-heurísticas individualmente, o MAS também foi executado 10 vezes.

Tabela 4.2: Resultados obtidos pelo MAS na instância RC201.

Configuração	**Tempo Médio (s)**	**Média K**	**Média D**	**Melhor Solução**
MAS (10 execuções)	$305,09 \pm 18,08$	$12,60 \pm 1,07$	$3365,64 \pm 125,95$	$K = 11, D = 3374,26$

Para melhor visualização e compreensão, a melhor execução encontrada dentre as 10 testadas é analisada individualmente a seguir.

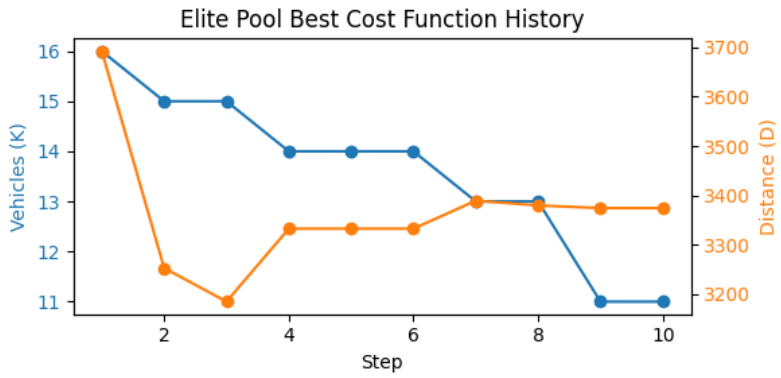

Figura 4.3: Evolução da função de custo global ao longo das iterações (*steps*) do MAS. Os valores representam o desempenho da melhor solução armazenada na memória compartilhada a cada ciclo

Abaixo, o registro de eventos da execução do MAS mostra o comportamento e evolução a cada passo.

```
Mesa MAS Step 1/10
Agent TS result -> vehicles: 22, distance: 3783.54
Agent GA result -> vehicles: 21, distance: 4153.16
Agent SA result -> vehicles: 16, distance: 3692.41
==> PR start: source (k=16, d=3692.41) -> target (k=21, d=4153.16), diversity=99
    PR candidates -> forward (k=16, d=3692.41), reverse (k=16, d=3692.41)
    PR finished: no improvement over source

Mesa MAS Step 2/10
Agent TS result -> vehicles: 16, distance: 3633.64
Agent SA result -> vehicles: 15, distance: 3272.03
Agent GA result -> vehicles: 15, distance: 3272.03
==> PR start: source (k=15, d=3272.03) -> target (k=21, d=4153.16), diversity=100
    PR candidates -> forward (k=15, d=3252.67), reverse (k=15, d=3272.03)
    PR improved elite pool -> (k=15, d=3252.67)

Mesa MAS Step 3/10
Agent SA result -> vehicles: 15, distance: 3245.89
```

```
Agent GA result -> vehicles: 15, distance: 3252.67
Agent TS result -> vehicles: 16, distance: 3635.49
==> PR start: source (k=15, d=3245.89) -> target (k=16, d=3633.64), diversity=91
    PR candidates -> forward (k=15, d=3184.56), reverse (k=15, d=3245.89)
    PR improved elite pool -> (k=15, d=3184.56)

...

Mesa MAS Step 9/10
Agent GA result -> vehicles: 13, distance: 3379.57
Agent SA result -> vehicles: 11, distance: 3374.26
Agent TS result -> vehicles: 13, distance: 3389.28
==> PR start: source (k=11, d=3374.26) -> target (k=14, d=3298.28), diversity=97
    PR candidates -> forward (k=11, d=3374.26), reverse (k=11, d=3374.26)
    PR finished: no improvement over source

Mesa MAS Step 10/10
Agent TS result -> vehicles: 13, distance: 3389.28
Agent SA result -> vehicles: 12, distance: 3521.73
Agent GA result -> vehicles: 12, distance: 3521.73
==> PR start: source (k=11, d=3374.26) -> target (k=12, d=3521.73), diversity=94
    PR candidates -> forward (k=11, d=3374.26), reverse (k=11, d=3374.26)
    PR finished: no improvement over source


Mesa MAS runtime: 302.4375 seconds
Mesa MAS best -> vehicles: 11, distance: 3374.26
```

4.3.4 Análise dos resultados

Ao observar o registro de eventos (*logs*), nota-se que o *Path Relinking* nem sempre obtém melhorias imediatas e, quando as encontra, estas consistem em pequenas reduções no objetivo secundário (distância D). Esse comportamento sugere que a estratégia de cooperação atual pode ser aprimorada, seja através de uma nova dinâmica de interação entre os agentes ou pela implementação de variações do PR, como o PR evolutivo ou truncado.

A análise conjunta da Tabela 4.2 e da Tabela 4.1 indica que o MAS obteve resultados competitivos dentro de um tempo de execução similar ao das meta-heurísticas isoladas. O sistema foi capaz de encontrar uma solução com $K = 11$ veículos, igualando-se ao desempenho de pico do SA e do TS, embora com uma distância total (D) superior à desses modelos.

No entanto, os desvios padrão de K e D ligeiramente maiores no MAS sugerem que a interação entre agentes heterogêneos via *ElitePool* e *Path Relinking* introduz uma variabilidade que o sistema ainda não estabilizou. Aumentar o tempo de execução e o número de ciclos de cooperação poderia ser benéfico para permitir um refinamento mais profundo das soluções compartilhadas.

Não obstante, a Figura 4.3 demonstra que o MAS também enfrenta o desafio de *trade-off* observado anteriormente: o aumento da distância D ocorre sempre que há uma reconfiguração drástica das sub-rotas para a redução do número de veículos (K). Esse fenômeno reforça a natureza multiobjetivo do *VRPTW* e a dificuldade em otimizar ambos os indicadores simultaneamente em cenários de alta restrição.

4.4 Hiper-heurística baseada em Aprendizado por Reforço

A evolução do sistema multiagente convencional para o modelo proposto reside na implementação da classe `VRPOptimizationModelRL`. Neste modelo, o mecanismo de coordenação deixa de ser puramente aleatório e passa a utilizar o algoritmo *Q-Learning* para orquestrar a ativação dos agentes especialistas, configurando uma abordagem de hiper-heurística baseada em aprendizado [15].

4.4.1 Modelagem do Q-Learning

O agente de RL opera sobre um ciclo de percepção e ação, onde a política de decisão é refinada com base na experiência acumulada no ambiente de busca. Os componentes do modelo são definidos como segue:

- **Espaço de Estados (S):** O estado é definido pela tupla discreta (K, D_{bin}). Para evitar a explosão do espaço de estados e garantir a convergência da Tabela-Q, a distância total (D) é discretizada em faixas ($bins$) de 100 unidades através do parâmetro `distance_bin_size`.

- **Espaço de Ações (A):** Corresponde à seleção de uma das três meta-heurísticas

disponíveis para execução no ciclo atual: *Simulated Annealing* (SA), *Tabu Search* (TS) ou *Genetic Algorithm* (GA).

- **Função de Recompensa Hierárquica (R):** A recompensa é estruturada para refletir a prioridade multiobjetivo do *VRPTW*:
  - **Redução de Frota:** Atribui-se uma recompensa massiva (+200) caso a ação resulte na diminuição do número de veículos (K).
  - **Redução de Distância:** Caso K se mantenha constante, a recompensa é equivalente à magnitude da melhoria em D.
  - **Penalidade por Estagnação:** Aplica-se uma pequena penalidade (-1) se a ação não gerar melhorias, incentivando a exploração de outros agentes.
- **Política de Seleção:** Utiliza-se a estratégia ϵ-*greedy* com $\epsilon = 0,4$, equilibrando a explotação do conhecimento atual com a exploração de novas trajetórias de busca.

4.4.2 Dinâmica de Cooperação e Atualização

A cada *step* do modelo, o sistema realiza três iterações de escolha e execução. Após a execução do agente selecionado, o valor de utilidade $Q(s, a)$ é atualizado na Tabela-Q utilizando a Equação 2.3:

Os hiperparâmetros adotados incluem uma taxa de aprendizado $\alpha = 0,2$ e um fator de desconto $\gamma = 0,8$, priorizando a importância de recompensas a curto e médio prazo no processo de otimização. Paralelamente ao RL, o método `run_path_relinking` continua a promover a síntese de soluções de elite, agora alimentado por um *pool* cujos candidatos são gerados por agentes selecionados de forma adaptativa.

4.4.3 Resultados

Tabela 4.3: Resultados obtidos pelo MAS-RL na instância RC201

Configuração	**Tempo Médio (s)**	**Média K**	**Média D**	**Melhor Solução**
MAS-RL (10 execuções)	$313,17 \pm 8,96$	$13,20 \pm 0,92$	$3342,14 \pm 169,01$	$K = 11, D = 3393,62$

Novamente, a evolução da função de custo e o registro de execução para detalhamento, utilizando uma execução como exemplo. Desta vez, para o MAS-RL.

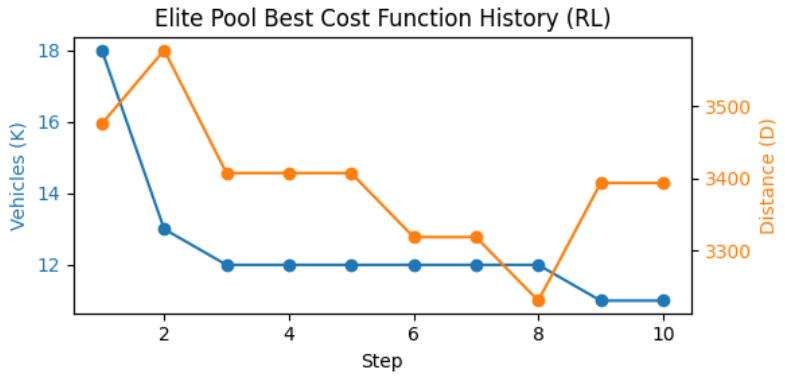

Figura 4.4: Evolução da função de custo global ao longo das iterações do MAS-RL

```
--- MAS RL SINGLE RUN (seed=42) ---

MAS RL Step 1/10
==> RL Run 1/3 | State (999, 999) -> Selected Action: SA
    Agent SA step result -> vehicles: 18, distance: 3475.35
    Update: Reward=0.00 | Q(SA) of state(999, 999) updated to 0.00
==> RL Run 2/3 | State (18, 34) -> Selected Action: TS
    Agent TS step result -> vehicles: 18, distance: 3475.35
    Update: Reward=-1.00 | Q(TS) of state(18, 34) updated to -0.20
==> RL Run 3/3 | State (18, 34) -> Selected Action: GA
    Agent GA step result -> vehicles: 18, distance: 3475.35
    Update: Reward=-1.00 | Q(GA) of state(18, 34) updated to -0.20
==> PR start: source (k=18, d=3475.35) -> target (k=18, d=3475.35), diversity=0
    PR candidates -> forward (k=18, d=3475.35), reverse (k=18, d=3475.35)
    PR finished: no improvement over source

MAS RL Step 2/10
==> RL Run 1/3 | State (18, 34) -> Selected Action: SA
    Agent SA step result -> vehicles: 15, distance: 3534.34
    Update: Reward=141.01 | Q(SA) of state(18, 34) updated to 28.20
==> RL Run 2/3 | State (15, 35) -> Selected Action: TS
    Agent TS step result -> vehicles: 15, distance: 3534.34
    Update: Reward=-1.00 | Q(TS) of state(15, 35) updated to -0.20
==> RL Run 3/3 | State (15, 35) -> Selected Action: SA
    Agent SA step result -> vehicles: 13, distance: 3577.39
    Update: Reward=156.95 | Q(SA) of state(15, 35) updated to 31.39
==> PR start: source (k=13, d=3577.39) -> target (k=18, d=3475.35), diversity=98
    PR candidates -> forward (k=13, d=3577.39), reverse (k=13, d=3577.39)
    PR finished: no improvement over source
```

```
MAS RL Step 3/10
==> RL Run 1/3 | State (13, 35) -> Selected Action: SA
    Agent SA step result -> vehicles: 12, distance: 3415.97
    Update: Reward=361.41 | Q(SA) of state(13, 35) updated to 72.28
==> RL Run 2/3 | State (12, 34) -> Selected Action: SA
    Agent SA step result -> vehicles: 17, distance: 3523.85
    Update: Reward=-1.00 | Q(SA) of state(12, 34) updated to -0.20
==> RL Run 3/3 | State (12, 34) -> Selected Action: TS
    Agent TS step result -> vehicles: 17, distance: 3501.73
    Update: Reward=-1.00 | Q(TS) of state(12, 34) updated to -0.20
==> PR start: source (k=12, d=3415.97) -> target (k=17, d=3501.73), diversity=100
    PR candidates -> forward (k=12, d=3415.97), reverse (k=12, d=3407.32)
    PR improved elite pool -> (k=12, d=3407.32)

...

MAS RL Step 9/10
==> RL Run 1/3 | State (12, 32) -> Selected Action: SA
    Agent SA step result -> vehicles: 11, distance: 3393.62
    Update: Reward=37.25 | Q(SA) of state(12, 32) updated to 7.45
==> RL Run 2/3 | State (11, 33) -> Selected Action: SA
    Agent SA step result -> vehicles: 12, distance: 3092.67
    Update: Reward=-1.00 | Q(SA) of state(11, 33) updated to -0.20
==> RL Run 3/3 | State (11, 33) -> Selected Action: TS
    Agent TS step result -> vehicles: 12, distance: 3273.58
    Update: Reward=-1.00 | Q(TS) of state(11, 33) updated to -0.20
==> PR start: source (k=11, d=3393.62) -> target (k=12, d=3092.67), diversity=99
    PR candidates -> forward (k=11, d=3393.62), reverse (k=11, d=3393.62)
    PR finished: no improvement over source

MAS RL Step 10/10
==> RL Run 1/3 | State (11, 33) -> Selected Action: GA
    Agent GA step result -> vehicles: 12, distance: 3230.87
    Update: Reward=-1.00 | Q(GA) of state(11, 33) updated to -0.20
==> RL Run 2/3 | State (11, 33) -> Selected Action: TS
    Agent TS step result -> vehicles: 12, distance: 3092.67
    Update: Reward=-1.00 | Q(TS) of state(11, 33) updated to -0.39
==> RL Run 3/3 | State (11, 33) -> Selected Action: SA
    Agent SA step result -> vehicles: 12, distance: 3273.58
    Update: Reward=-1.00 | Q(SA) of state(11, 33) updated to -0.39
==> PR start: source (k=11, d=3393.62) -> target (k=12, d=3092.67), diversity=99
    PR candidates -> forward (k=11, d=3393.62), reverse (k=11, d=3393.62)
    PR finished: no improvement over source


Run summary | runtime: 305.9531s | K: 11, D: 3393.62 | Q states: 8, Visited states:
     8
```

Neste modelo, podemos também utilizar a tabela-Q como objeto de análise, conjuntamente com a distribuição de execuções de cada agente, que não é mais uniforme.

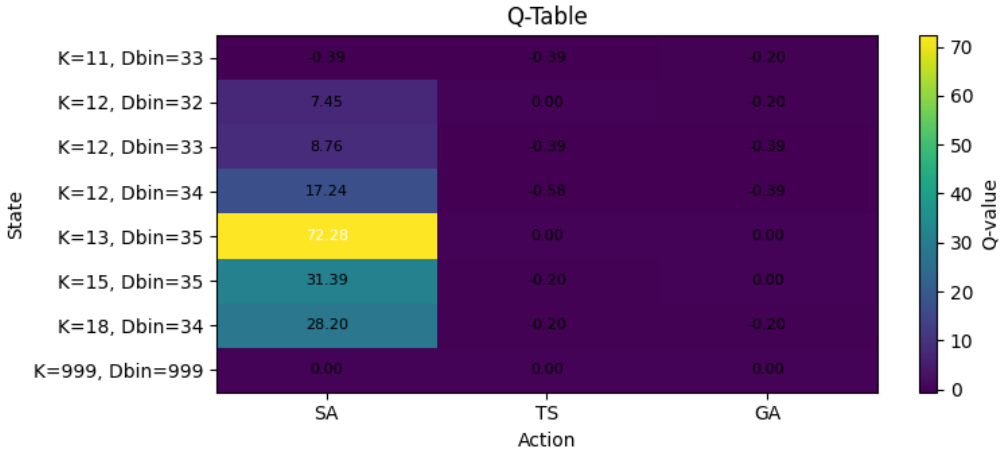

Figura 4.5: *Q-table* referente a execução do MAS-RL

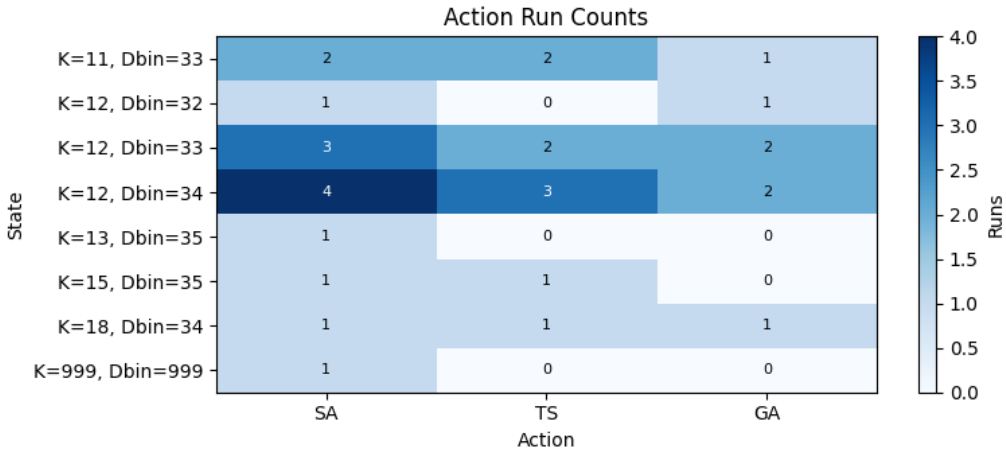

Figura 4.6: Distribuição do número de execuções das meta-heurísticas por estado

4.4.4 Análise dos Resultados

Ao analisar o desempenho do sistema, observa-se que, apesar de apresentar características de estabilidade e convergência similares ao modelo cooperativo base, o MAS-RL obteve resultados levemente inferiores. Este comportamento pode ser atribuído a múltiplos fatores técnicos, com destaque para a necessidade de um tempo de treinamento mais extenso para a convergência da Tabela-Q e a calibração fina dos hiperparâmetros de aprendizado. Diferente do sistema puramente cooperativo, que

dedica integralmente seu orçamento computacional à explotação, o modelo com RL despende recursos na fase de exploração através da política ϵ-greedy. Este custo de exploração, embora essencial para o mapeamento do espaço de estados, pode limitar a capacidade de refinamento intensivo das soluções no mesmo intervalo de tempo, uma vez que o agente ocasionalmente seleciona ações subótimas para consolidar seu conhecimento.

O registro de eventos demonstra que o sistema tratou todos os agentes de forma equivalente nos estágios iniciais, o que é coerente com a natureza do problema, dado que soluções pouco refinadas permitem melhorias significativas independentemente do algoritmo empregado. Observou-se que o sistema transita rapidamente entre diferentes configurações, o que resulta em uma permanência reduzida em estados específicos, dificultando o aprendizado profundo sobre qual meta-heurística seria superior em circunstâncias exatas de transição. Entretanto, à medida que a busca se torna mais complexa e o espaço de soluções viáveis se estreita, a dinâmica de seleção de ações altera-se, conferindo maior foco à Busca Tabu (TS) e, primordialmente, ao *Simulated Annealing* (SA). Este redirecionamento é esperado, visto que estas técnicas demonstram maior eficácia no refinamento local em instâncias de alta restrição temporal como as do VRPTW .

A eficácia do aprendizado é validada pela consistência entre as escolhas do agente de RL e os desempenhos individuais apresentados na Tabela 4.1, onde os algoritmos SA e TS obtiveram resultados superiores ao GA. Este alinhamento, visível tanto na Figura 4.6 quanto na Figura 4.5, confirma que o sistema foi capaz de identificar e priorizar as ferramentas mais robustas para as etapas críticas da otimização, demonstrando que houve uma evolução real na política de tomada de decisão. Além disso, ressalta-se que o custo computacional exigido para a manutenção e atualização da Tabela-Q é desprezível frente ao esforço de processamento das meta-heurísticas, o que valida a arquitetura sob a perspectiva de eficiência algorítmica e escalabilidade.

No que tange ao *trade-off* entre o número de veículos (K) e a distância total (D), inerente à natureza multiobjetivo do VRPTW, o sistema com RL demonstrou uma capacidade de suavização superior. Através da calibração do fator de desconto

(γ) e da função de recompensa (r), possivelmente o modelo aprendeu que reduções drásticas em K que resultam em uma degradação severa de D geram menores recompensas acumuladas a longo prazo. Esse aprendizado resultou na geração de rotas mais equilibradas, onde a busca pela economia de frota não sacrificou drasticamente a eficiência da distância. Este fenômeno sugere que a inteligência de controle via RL permite uma navegação mais estratégica no espaço de busca, mitigando os saltos de custo observados na abordagem puramente cooperativa e oferecendo soluções com maior viabilidade operacional.

Capítulo 5

Conclusão

Embora os resultados quantitativos obtidos pelo sistema MAS simples e pelo MAS-RL não tenham superado as métricas de desempenho das meta-heurísticas individuais, os experimentos permitiram extrair aprendizados fundamentais sobre a coordenação de meta-heurísticas. A integração do Aprendizado por Reforço validou a capacidade do sistema de mapear a utilidade de diferentes algoritmos, demonstrando que o desenvolvimento de arquiteturas híbridas complexas exige um equilíbrio sensível entre o tempo de aprendizado e a intensidade da explotação.

5.1 Resultados e análises finais

A análise dos dados revela que o sistema foi capaz de identificar a hierarquia de eficiência entre os agentes, com a distribuição de execuções refletindo fielmente o sucesso individual observado no SA e no TS frente ao GA. A teoria nos diz que a arquitetura de sistema multi-agente é uma solução eficaz para o problema de estagnação das meta-heurísticas isoladas. Enquanto o Algoritmo Genético pode convergir prematuramente, mas deixando soluções razoavelmente eficientes e distintas na memória compartilhada, o MAS permite que a Busca Tabu e o Recozimento Simulado refinem essas soluções de elite, enquanto a estratégia de *Path Relinking* constrói pontes entre diferentes regiões do espaço de busca, maximizando a exploração global do VRPTW.

Conclui-se, contudo, que ao menos a curto prazo, a aplicação de um único método reconhecidamente robusto pode ser mais efetiva do que um sistema híbrido MAS. Ao aplicar métodos não muito efetivos (principalmente em fases finais), como o GA, o MAS causa, evidentemente, um desperdício de recursos que poderiam ser usados com TS ou SA.

Por sua vez, a superioridade do SMA simples frente ao MAS-RL também reflete a dificuldade em calibrar parâmetros e a sensibilidade do Q-Learning, o que pode indicar, mais uma vez, que uma solução mais rebuscada não é necessariamente superior se não dispuser de ciclos de execução suficientes para compensar o custo inicial do aprendizado. O *trade-off* entre a sofisticação da arquitetura e o tempo computacional viável permanece como um desafio central, reforçando que a inteligência coletiva demanda um amadurecimento tático que o MAS simples, por sua natureza puramente estocástica, não necessita percorrer.

5.2 Oportunidades de melhoria

No âmbito do aprimoramento do trabalho em si, identificou-se a necessidade de evoluir a modelagem do estado no *Q-Learning*. Uma proposta promissora é a implementação de herança de estados, permitindo que parte do valor-Q de um estado anterior seja transmitido para o sucessor, acelerando a convergência do sistema. É possível, também, iniciar uma execução de otimização com uma tabela-Q já existente de um treinamento anterior, o que aceleraria a convergência e os resultados. Adicionalmente, a otimização dos parâmetros das meta-heurísticas pode ser realizada através de abordagens teóricas ou empíricas mais refinadas.

Quanto à expansão do trabalho e adições estruturais, sugere-se a hibridização profunda através de Algoritmos Meméticos, utilizando o conhecimento adquirido sobre o desempenho individual das meta-heurísticas para compensar fraquezas específicas de cada uma, como já realizado em parte pelo MAS. Para aumentar a validade estatística do modelo, é fundamental testar o sistema em múltiplos conjuntos de dados com diferentes distribuições geográficas e números de pontos, evitando o viés de sobreajuste à uma única instância do *Benchmark* de Solomon. Por fim, a aplicação

em casos reais [16], incorporando mapas georreferenciados e tempos de deslocamento não lineares, permitiria avaliar a robustez do SMA-RL frente às imprevisibilidades do transporte urbano, aproximando a pesquisa acadêmica das demandas práticas do setor de logística.

Outro ponto para o aumento da verossimilhança é o ajuste da função de custo para considerar explicitamente o tempo de espera, já que este fator representou uma parcela significativa do tempo total da rota (não a distância, que é medida) nos experimentos realizados, impactando diretamente na eficiência logística.

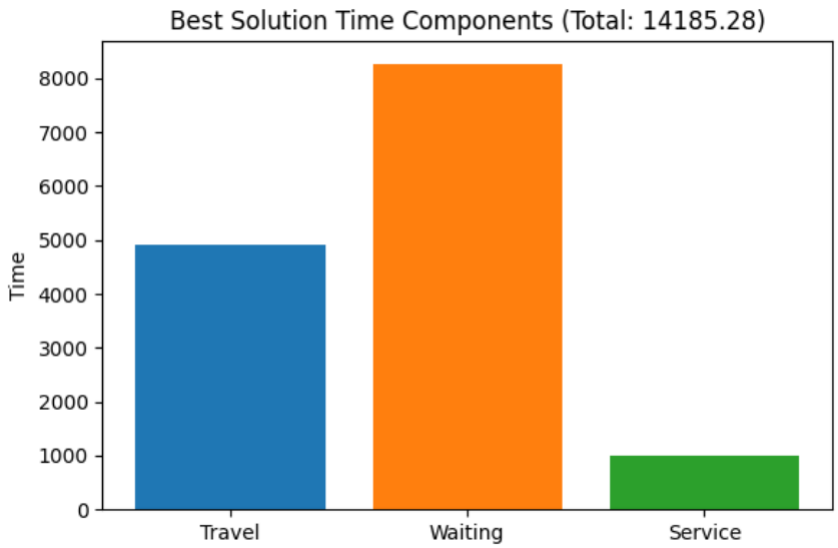

Figura 5.1: Tempos gastos com viagem, espera e serviço em uma execução simples de otimização

Note uma desproporção crítica entre o tempo de deslocamento e a ociosidade por espera, evidenciando ineficiências que impactam a viabilidade real das rotas. Visto que o tempo de serviço total é invariante e fixo (por ser a soma dos atendimentos individuais), a elevada espera indica chegadas prematuras às janelas de tempo. Esse cenário reforça o desafio de sincronização temporal do VRPTW, validando a necessidade de mecanismos de controle para mitigar a inatividade operacional.

Referências Bibliográficas

[1] GAREY, M. R., JOHNSON, D. S., *Computers and intractability: a guide to the theory of NP-completeness*. San Francisco, W. H. Freeman & Co., 1979.

[2] DANTZIG, G. B., RAMSER, J. H., "The truck dispatching problem", *Management Science*, v. 6, n. 1, pp. 80–91, 1959.

[3] SOLOMON, M. M., "Algorithms for the vehicle routing and scheduling problems with time window constraints", *Operations Research*, v. 35, n. 2, pp. 254–265, 1987.

[4] KIRKPATRICK, S., GELATT, C. D., VECCHI, M. P., "Optimization by simulated annealing", *Science*, v. 220, n. 4598, pp. 671–680, 1983.

[5] GLOVER, F., "Tabu search: part I", *ORSA Journal on Computing*, v. 1, n. 3, pp. 190–206, 1989.

[6] HOLLAND, J. H., *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. Ann Arbor, University of Michigan Press, 1975.

[7] GOLDBERG, D. E., *Genetic algorithms in search, optimization, and machine learning*. Reading, Addison-Wesley, 1989.

[8] WOOLDRIDGE, M., JENNINGS, N. R., "Intelligent agents: theory and practice", *The Knowledge Engineering Review*, v. 10, n. 2, pp. 115–152, 1995.

[9] Ter Hoeven, E., OTHERS, "Mesa 3: agent-based modeling with Python in 2025", *Journal of Open Source Software*, v. 10, n. 107, pp. 7668, 2025.

[10] GLOVER, F., LAGUNA, M., MARTÍ, R., "Fundamentals of scatter search and path relinking", *Control and Cybernetics*, v. 29, n. 3, pp. 653–684, 2000.

[11] SUTTON, R. S., BARTO, A. G., *Reinforcement learning: an introduction*. 2 ed. Cambridge, MIT Press, 2018.

[12] WATKINS, C. J. C. H., DAYAN, P., "Q-learning", *Machine Learning*, v. 8, n. 3-4, pp. 279–292, 1992.

[13] SINTEF, "Vehicle Routing Problem with Time Windows: 100 Customers", `https://www.sintef.no/projectweb/top/vrptw/100-customers/`, (Acesso em 06 de março de 2026).

[14] MESTER, D., BRÄYSY, O., DULLAERT, W., *A Multi-parametric Evolution Strategies Algorithm for Vehicle Routing Problems*, Working paper, Institute of Evolution, University of Haifa, Israel, 2005.

[15] BURKE, E. K., OTHERS, "Hyper-heuristics: a survey of the state of the art", *Journal of the Operational Research Society*, v. 64, n. 12, pp. 1695–1724, 2013.

[16] NVlabs, "olist-vrp-benchmark: Problem instances for the Vehicle Routing Problem", `https://github.com/NVlabs/olist-vrp-benchmark`, (Acesso em 06 de março de 2026).

Apêndice A

Benchmark de Solomon

No contexto de otimização e logística, as instâncias de *Solomon* constituem um conjunto de problemas de teste (*benchmarks*) propostos por Marius Solomon em 1987 para o VRPTW. Essas instâncias são fundamentais para aferir a eficiência de algoritmos, pois representam diferentes desafios geográficos e temporais. As nomenclaturas **C**, **R** e **RC** referem-se à disposição espacial dos clientes:

- **Tipo C (*Clustered*):** Os clientes estão organizados em agrupamentos (*clusters*). O algoritmo deve ser eficiente em atender aos clientes de um grupo denso antes de se deslocar para o próximo. Geralmente, este perfil é considerado o de mais fácil resolução para a obtenção de rotas ótimas.

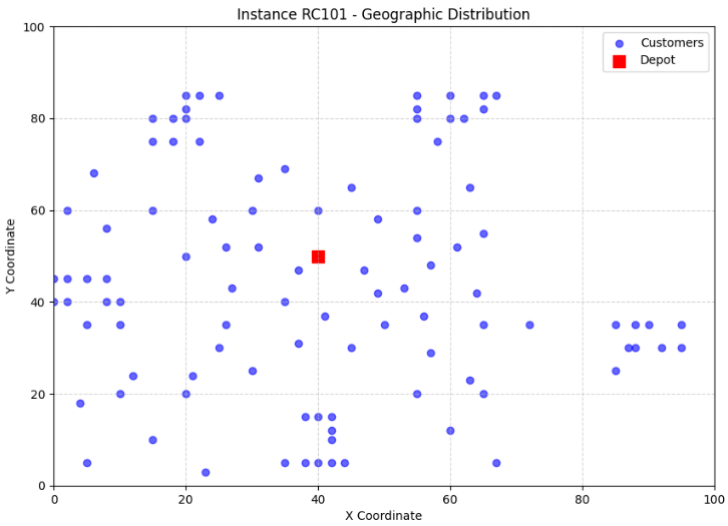

Figura A.1: Distribuição geográfica dos clientes na instância *Clustered* C101

- **Tipo R (*Random*):** Os clientes estão distribuídos de forma aleatória e uni-

forme no plano de coordenadas. A ausência de padrões óbvios exige que o algoritmo avalie criteriosamente as distâncias e as janelas de tempo para cada tomada de decisão.

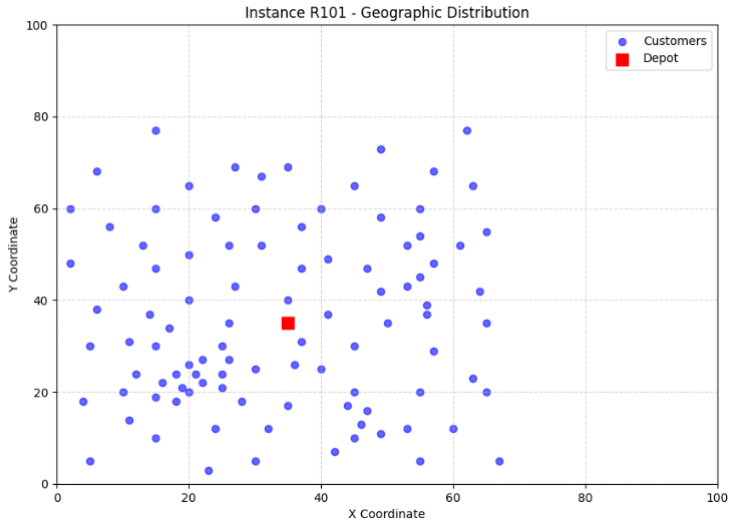

Figura A.2: Distribuição geográfica dos clientes na instância *Random* R101

- **Tipo RC (*Random-Clustered*):** Trata-se de um modelo híbrido que combina regiões de distribuição aleatória com agrupamentos definidos. Este cenário aproxima-se da realidade urbana complexa, onde centros comerciais densos convivem com áreas de entregas residenciais esparsas.

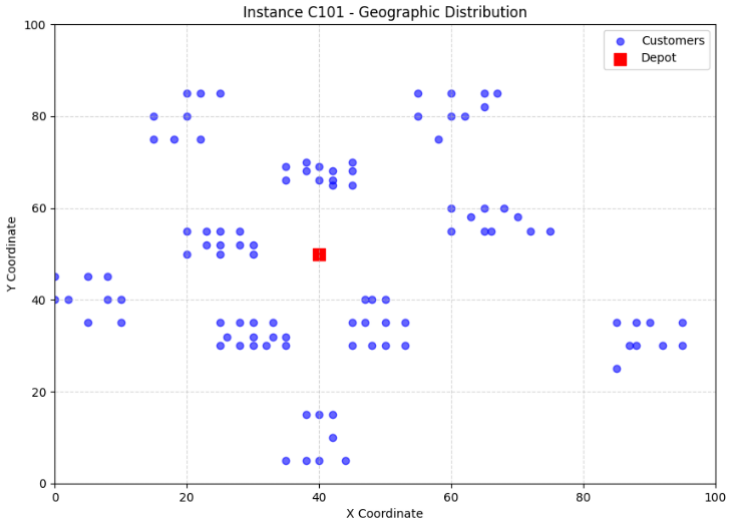

Figura A.3: Distribuição geográfica dos clientes na instância *Random-Clustered* RC101

Adicionalmente, as séries **1** e **2** definem o rigor das restrições operacionais, como a capacidade de carga dos veículos e a amplitude das janelas de tempo:

- **Série 1:** Apresenta restrições severas, com veículos de baixa capacidade de carga e janelas de tempo estreitas. Tais limitações resultam em rotas curtas e exigem uma frota maior para cobrir todos os pontos. Como exemplo, a instância *RC101* possui uma janela de tempo média de apenas $32,08$.

- **Série 2:** Oferece um cenário mais flexível, com veículos de grande capacidade e janelas de tempo expandidas. Um único veículo consegue atender a uma quantidade maior de clientes, transferindo o desafio principal para a minimização da distância total percorrida. A instância *RC201*, utilizada nos testes deste trabalho, possui uma janela média significativamente superior, de $128,32$.

Apêndice B

Estimativa para Busca Exaustiva

Este apêndice dedica-se a estimar o tempo necessário para a obtenção de uma solução ótima, por força bruta, referente à instância RC201, a qual contém 100 clientes (incluindo o depósito). O objetivo fundamental desta seção é evidenciar a natureza combinatória da complexidade temporal intrínseca a problemas de otimização de rotas *NP-hard* [1], como o *VRP*. Mesmo adotando a simplificação extrema do cenário para uma formulação análoga ao clássico Problema do Caixeiro Viajante (*TSP*), onde um único veículo visita sequencialmente todos os destinos, as permutações resultantes exigem um esforço computacional assintoticamente proibitivo.

O número de sequências possíveis para um veículo visitar todos os pontos e retornar à origem é dado por:

$$99! \approx 9,3326 \times 10^{155}$$

Para este cenário, assumimos um supercomputador teórico capaz de avaliar 1 bilhão (10^9) de soluções por milissegundo. Calculando a vazão anual desta máquina:

$$10^{12} \times 60 \times 60 \times 24 \times 365,25 \approx 3,15 \times 10^{19} \text{ sol./ano}$$

O tempo estimado para encontrar o ótimo absoluto seria:

$$\text{Tempo} = \frac{9,33 \times 10^{155}}{3,15 \times 10^{19}} \approx 2,96 \times 10^{136} \text{ anos}$$

Para compreender a magnitude deste resultado, a idade estimada do universo é de aproximadamente $13,8 \times 10^9$ anos. Outra comparação é que há "apenas" 10^{80} átomos

no universo observável. Se tivéssemos começado o processamento no momento do *Big Bang*, até o presente momento teríamos percorrido uma fração tão pequena do espaço de busca que ela seria estatisticamente equivalente a 0%.